DLOOP: LOOPED SPECULATIVE DECODING

Anonymous authors
Paper under double-blind review

ABSTRACT

Speculative decoding accelerates autoregressive generation in large language models. In each drafting stage, a lightweight draft model proposes tokens that the target model subsequently verifies. With increasingly capable draft models, we find that the target model frequently accepts all tokens produced in a drafting stage. A verification nevertheless follows each drafting stage, resulting in unnecessary target-model forward passes even when drafting could have continued. Adaptive draft length methods decide during decoding how many draft tokens precede a verification, but they raise the speedup only for autoregressive draft models. For a parallel draft model, drafting further requires target-model hidden states for draft tokens that have not been verified. We propose **DLoop**, a looped form of speculative decoding that adaptively performs multiple drafting stages before verification. DLoop continues drafting while the draft model remains confident and verifies all accumulated draft tokens together. Loop-aware training keeps the draft model reliable in the additional drafting stages by exposing it to its own hidden states for unverified draft tokens. By spending additional draft-model forward passes, DLoop reduces the number of target-model forward passes required for verification. Across diverse speculative decoding methods including EAGLE-3, DFlash, Domino, DSpark, and multi-token prediction modules, DLoop improves the wall-clock speedup by 5 to 41 percent while preserving lossless decoding.

1 INTRODUCTION

Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) accelerates autoregressive generation in large language models (Brown et al., 2020; Achiam et al., 2023; Team et al., 2023; Grattafiori et al., 2024; Liu et al., 2024a; Yang et al., 2025), where each token costs a full forward pass of the model. In each drafting stage, a lightweight draft model proposes several draft tokens that the target model is likely to generate next. It produces them either token by token (Li et al., 2024a;b; 2025) or in parallel (Chen et al., 2026; Cheng et al., 2026; Huang et al., 2026). The target model then verifies these draft tokens in a single forward pass while preserving its output distribution.

Speculative decoding performs a verification after every drafting stage, even when the draft model could have continued. If the draft model keeps producing tokens that the target model accepts, additional lightweight drafting stages can replace costly verifications. As shown in Fig. 1, across four draft models and three task categories, a substantial fraction of drafting stages are fully accepted, that is, every draft token passes verification. Verification also dominates decoding time, since it requires a full forward pass of the target model. The latency of that pass, however, grows only modestly with the number of tokens it verifies (Pope et al., 2023).

Adaptive draft length methods were proposed to address this problem by deciding during decoding how many draft tokens precede a verification (Kim et al., 2023; Zhang et al., 2025; Agrawal et al., 2024; Xu et al., 2023; Huang et al., 2025; Mamou et al., 2024; Zhang et al., 2026; 2024b; Brown et al., 2024). These methods raise the speedup only for autoregressive draft models as shown in Section 4.2. These methods build on an autoregressive draft model, either a separate small language model or a drafter of the EAGLE family (Li et al., 2024a;b; 2025). Such a draft model produces one token at a time and can therefore stop or continue drafting at any token. A parallel draft model (Chen et al., 2026; Huang et al., 2026; Cheng et al., 2026) instead produces all draft tokens of a drafting stage in a single forward pass. Stopping early leaves the cost of this forward pass unchanged. Drafting further requires target-model hidden states for draft tokens that have not been verified. Without them, the draft model conditions on its own hidden states, a setting it was not trained for.

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

In this work, we propose **DLoop**, a looped form of speculative decoding in which a single verification covers multiple drafting stages. DLoop consists of two components, a *confidence-gated drafting loop* at inference and *loop-aware training*. The drafting loop, illustrated in Fig. 2, decides how many drafting stages a single verification covers. It repeats the drafting stage while the draft model remains confident in the draft tokens it has just produced. After each drafting stage, the loop scores this confidence by summing the log probabilities of these draft tokens. If the score remains above a threshold, the loop performs an additional drafting stage. Otherwise, the target model verifies all accumulated draft tokens in one forward pass. Since the loop repeats whole drafting stages, it applies to parallel and autoregressive draft models alike. As in standard speculative decoding, the acceptance rule is unchanged and decoding remains lossless.

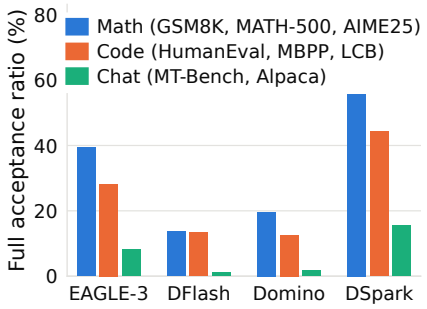


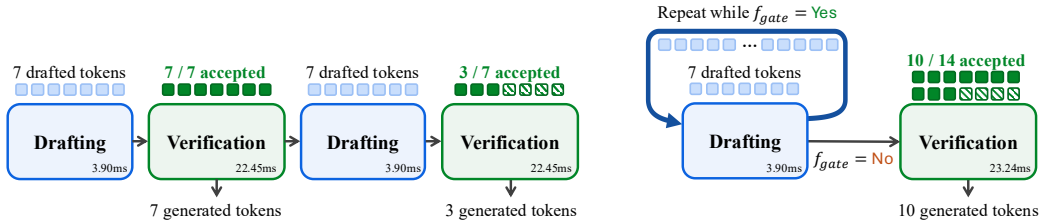
Figure 1: **Full acceptance is common.** Fraction of drafting stages in which the target model accepts every draft token, under standard speculative decoding with Qwen3-4B (Yang et al., 2025) and greedy decoding ($T = 0$).

Loop-aware training aims to keep the draft model reliable in the additional drafting stages. An additional drafting stage conditions on the draft tokens of earlier drafting stages. Target-model hidden states exist only for the tokens before the first drafting stage. Loop-aware training exposes the draft model to its own hidden states for the draft tokens that follow. It is inspired by the training-time test of EAGLE-3 (Li et al., 2025). That test unrolls the draft model over its own predicted features within one drafting stage. Loop-aware training extends this unrolling across the drafting stages of the loop. Unlike the training-time test, it thus applies to parallel draft models as well.

We evaluate DLoop on EAGLE-3 (Li et al., 2025), DFlash (Chen et al., 2026), Domino (Huang et al., 2026), DSpark (Cheng et al., 2026), and multi-token prediction modules (MTP) (Team, 2026a; Team et al., 2026). In every one of these settings, the loop and loop-aware training together raise both the acceptance length and the wall-clock speedup. DLoop obtains this speedup by spending additional draft-model forward passes to reduce the number of target-model forward passes. It does not require additional parameters and applies on top of these publicly released draft models. DLoop does not change how a draft model produces tokens, only how many drafting stages precede a verification. The number of drafting stages before a verification need not be fixed by the schedule and can be decided during decoding.

2 Related Work

Speculative decoding. The speedup of speculative decoding (Leviathan et al., 2023; Chen et al., 2023) depends on the acceptance length and the cost of a drafting stage. Early formulations train a separate draft model from the same family as the target model, whereas later work builds the draft model on the target model itself. Decoding heads attached to the target model (Stern et al., 2018; Cai et al., 2024) propose several future tokens in parallel from its already computed hidden states. The EAGLE family (Li et al., 2024a;b; 2025) replaces these parallel heads with a lightweight autoregressive drafter that conditions on the same hidden states. The multi-token prediction modules released with recent models (Team, 2026a; Team et al., 2026) take the same autoregressive form, feeding each hidden state back to produce the next draft token. EAGLE-3 (Li et al., 2025) trains its drafter with a training-time test, which unrolls it over its own predicted features within one drafting stage. Loop-aware training extends that unrolling across the drafting stages that a single verification covers. VAT (Gu et al., 2026) instead reshapes the supervision itself, weighting each position by whether it survives a simulated verification. The parallel draft models DFlash (Chen et al., 2026), Domino (Huang et al., 2026), and DSpark (Cheng et al., 2026) remove the sequential cost of autoregressive drafting by producing a whole block in one forward pass. Tree verification broadens what one drafting stage proposes to a tree of candidates that the target model verifies together (Miao et al., 2024; Li et al., 2024b; Ringel & Romano, 2026; Li et al., 2026). All of them change how a draft model produces a block, whereas DLoop changes only how many blocks precede a verification. The same loop therefore runs on EAGLE-3, DFlash, Domino, DSpark, and the multi-token prediction modules. It also composes with tree verification.



(a) Standard speculative decoding (b) Looped speculative decoding

Figure 2: **Comparison of speculative and looped speculative decoding.** We assume that the target model accepts all 7 tokens of the first drafting stage and 3 of 7 from the second, 10 tokens in total. The bonus token that each verification produces is omitted for clarity. (a) Speculative decoding verifies after each drafting stage and needs two verifications. (b) Looped speculative decoding skips the first verification because the gate function f_{gate} finds the first drafting stage confident. It then verifies all 14 draft tokens at once and yields the 10 tokens in a single verification. Latency is measured with a 1,024-token input prefix on a single H100 GPU, using Qwen3-4B (Yang et al., 2025) as the target model and DSpark (Cheng et al., 2026) as the draft model.

Adaptive draft length. Most speculative decoding methods fix the length of a drafting stage in advance, whereas prior works have explored determining its length dynamically during decoding. BiLD (Kim et al., 2023) and Draft&Verify (Zhang et al., 2024a) stop on the top-1 probability of the draft distribution. SVIP (Zhang et al., 2025) and AdaEDL (Agrawal et al., 2024) stop on its entropy. Dynamic depth decoding (DDD) (Brown et al., 2024) grows the draft tree (Li et al., 2024b) deeper while the summed probability of the sequences in the beam remains above a threshold. LLMCad (Xu et al., 2023) accumulates the confidences of the draft tokens within one drafting stage and calls the target model once the accumulated confidence falls below a threshold. Our gate function also accumulates confidence over a block, but it decides whether a further drafting stage should follow. Other methods train a module for the decision. SpecDec++ (Huang et al., 2025) predicts the acceptance probability of each draft token, DISCO (Mamou et al., 2024) classifies whether to continue, and DSpark (Cheng et al., 2026) sets a load-aware verification length from calibrated acceptance probabilities. AdaEAGLE (Zhang et al., 2024b) predicts the length once, before the drafting stage begins. All of them choose where the draft for one verification ends (for DSpark, how much of it is verified), whereas DLoop keeps a full drafting stage as the minimum and decides whether another drafting stage follows. PACER (Zhang et al., 2026) drafts autoregressively in blocks and appends another block while a separately trained pre-verification layer's mean predicted acceptance for the latest block stays above a threshold. Unlike PACER, DLoop gates on the confidence of the drafter itself and applies loop-aware training to the drafter. SpecBlock (Shi et al., 2026) trains a new draft model that drafts up to a fixed number of blocks before a verification, and a co-trained rank head decides which positions start a later block. DLoop instead applies to existing draft models without an additional module.

3 Looped Speculative Decoding

3.1 Background

Preliminary. Let $\mathcal{M}_t$ denote the target model, $\mathcal{M}_d$ the draft model, and k the block size, the number of draft tokens proposed in one drafting stage. Let S denote the loop depth, the number of drafting stages that a single verification covers. As shown in Fig. 2(a), standard speculative decoding runs one drafting stage ($S = 1$) and one verification. Drafting starts from a prefix $x_{1:t}$, the input prompt followed by the tokens verified so far, together with two quantities that $\mathcal{M}_t$ has produced for it. The first is the *anchor token* x_t, the last token of the prefix, from which the next block is predicted. It is the token that $\mathcal{M}_t$ itself produced in the preceding forward pass, commonly referred to as the *bonus token*. We call it the anchor token when a drafting stage starts from it and the bonus token when a verification produces it. The second is the target-side context, the hidden states $h_{1:t-1}$ gathered from a set of layers of $\mathcal{M}_t$, which $\mathcal{M}_d$ projects to its own hidden size. The context ends at position $t-1$ because $\mathcal{M}_t$ has generated the anchor token by next-token prediction but has not yet run a forward pass on it.

In the drafting stage, $\mathcal{M}_d$ conditions on the two quantities and proposes a block of k draft tokens,

$$\hat{x}_{t+1:t+k} = \mathcal{M}_d(h_{1:t-1}, x_t), \tag{1}$$

each with a log probability under $\mathcal{M}_d$. Existing draft methods differ in how the context is injected and in how the k tokens are produced. An autoregressive draft model (Li et al., 2025; Team, 2026a; Team et al., 2026) produces them one at a time and drafts each token after the first from its own hidden states. A parallel draft model (Chen et al., 2026; Huang et al., 2026; Cheng et al., 2026) instead produces all k tokens in a single forward pass.

In the verification, $\mathcal{M}_t$ processes the block in a single forward pass and applies the acceptance rule of speculative sampling (Leviathan et al., 2023; Chen et al., 2023). If the first $a \in \{0, \dots, k\}$ draft tokens are accepted, the verification adds these a tokens to the prefix together with a bonus token produced by $\mathcal{M}_t$. The hidden states of these positions come from the same pass. The token produced by $\mathcal{M}_t$ becomes the anchor token of the next drafting stage.

Limitation of fixed verification procedure. We argue that the de facto standard draft-verify procedure is inherently inefficient due to its fixed alternating structure. Specifically, even when all k draft tokens are accepted, the next drafting stage must wait for another verification. Thus, one verification yields at most $k+1$ tokens regardless of how accurately $\mathcal{M}_d$ drafts. Fig. 1 shows that a substantial fraction of blocks reach this bound. This bound is costly because verification requires a full forward pass of the target model and dominates decoding time. Adaptive draft length methods (Kim et al., 2023; Zhang et al., 2025; Huang et al., 2025; Cheng et al., 2026) decide during decoding how many draft tokens precede a verification. Section 4.2 shows that they raise the speedup only for autoregressive draft models. For a parallel draft model, stopping early leaves the cost of a drafting stage unchanged. Drafting further requires target-model hidden states for draft tokens that have not been verified. Our goal is to allow a single verification to cover multiple drafting stages.

3.2 Confidence-Gated Drafting Loop

DLoop performs $S \geq 1$ drafting stages and a single verification, as shown in Fig. 2(b). The s-th drafting stage produces a block $\hat{x}_{t+(s-1)k+1:t+sk}$. After each drafting stage, a gate function decides whether to perform another. When the gate function stops the loop, the target model verifies the accumulated draft tokens $\hat{x}_{t+1:t+Sk}$ in one forward pass. The gate function decides the number of drafting stages S during decoding. Since the loop repeats whole drafting stages, it applies to parallel and autoregressive draft models alike. Under full acceptance, one verification yields at most $Sk+1$ tokens, whereas S separate verifications yield at most $S(k+1)$. DLoop therefore forgoes up to $S-1$ intermediate bonus-token opportunities in exchange for the $S-1$ target-model forward passes it does not perform.

Gate function. Whether a block is fully accepted is known only after verification. The gate function therefore predicts from the draft distribution whether the latest block will be fully accepted. For each draft token $\hat{x}_i$ of the block just produced, we take the log probability ℓ_i that $\mathcal{M}_d$ assigns to it. Since the probability is normalized over the vocabulary, ℓ_i measures how much mass $\mathcal{M}_d$ places on $\hat{x}_i$ among all tokens at that position. The score c_s of the s-th block is the sum of these log probabilities, and the gate function compares it with a threshold g,

$$c_s = \sum_{i=t+(s-1)k+1}^{t+sk} \ell_i, \qquad f_{\text{gate}}(c_s) = \begin{cases} 1 & \text{if } c_s \geq g, \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

The sum corresponds to the product of the per-token probabilities, a quantity prior work uses to rank draft candidates (Li et al., 2024b) and to set the depth of a drafting stage (Brown et al., 2024). Since every ℓ_i is non-positive, a confident position cannot offset an uncertain one. The sum therefore falls whenever a single position is uncertain, in line with the acceptance rule that discards every token after the first rejection. DLoop performs another drafting stage while $f_{\text{gate}}(c_s) = 1$. Because the log probabilities ℓ_i are already computed in the drafting stage, the gate function adds only the summation and one comparison, without an additional module or forward pass. The score considers only the latest block, since reaching the s-th drafting stage already implies that every preceding block passed the gate. The threshold g is the only hyperparameter of the loop and is fixed before decoding.

Verification. The verification applies the acceptance rule of Section 3.1 to the accumulated draft tokens as if they were a single block of Sk tokens. Since the rule preserves the output distribution of $\mathcal{M}_t$ for a draft sequence of any length, decoding remains lossless (Leviathan et al., 2023; Chen et al., 2023). If a token in an earlier block is rejected, the verification also discards every later block. The drafting stages that produced them then add cost without adding accepted tokens. The accepted positions receive their hidden states h from the same pass. The next loop starts from the new anchor token as in standard speculative decoding.

Input to the additional drafting stages. The first drafting stage takes the target-side context $h_{1:t-1}$ and the anchor token x_t, as in Section 3.1. The additional drafting stages lack this input, since the draft tokens $\hat{x}_{t+1:t+(s-1)k}$ are not yet verified and $\mathcal{M}_t$ has not computed hidden states for them. We instead use the hidden states of $\mathcal{M}_d$. While producing a block, $\mathcal{M}_d$ computes its own final-layer hidden states $\tilde{h}$ at the positions it has already processed. The last draft token of the previous block takes the role of the anchor token for the s-th drafting stage. For $s \geq 2$,

$$\hat{x}_{t+(s-1)k+1:t+sk} = \mathcal{M}_d\big(h_{1:t-1} \,\|\, \tilde{h}_{t:t+(s-1)k-1},\, \hat{x}_{t+(s-1)k}\big), \tag{3}$$

where $\|$ denotes concatenation along the position dimension. The context is the target-side hidden states of the prefix followed by the draft-side hidden states from the first anchor token onward. It ends one position before the anchor token of the s-th drafting stage, as in standard speculative decoding. The hidden states $\tilde{h}$ occupy the positions that h would fill after verification. Each drafting stage therefore processes only its k new positions. The anchor token of the first drafting stage is produced by $\mathcal{M}_t$ and is always kept. The anchor token of an additional drafting stage is a draft token that the gate function has judged confident, but $\mathcal{M}_t$ has not yet verified. A parallel draft model (Chen et al., 2026; Huang et al., 2026; Cheng et al., 2026), however, never receives $\tilde{h}$ as input during training. An autoregressive draft model (Li et al., 2025; Team, 2026a; Team et al., 2026) receives $\tilde{h}$ only within a block. Section 3.3 describes how we train the draft model to take $\tilde{h}$ as input.

3.3 Loop-Aware Training

The draft models (Li et al., 2025; Chen et al., 2026; Cheng et al., 2026; Huang et al., 2026; Team, 2026a; Team et al., 2026) we build on are trained for a single drafting stage before each verification. During training, the first drafting stage receives the hidden states $h_{1:t-1}$ that $\mathcal{M}_t$ computed on a prefix and is supervised to predict the k tokens that follow. EAGLE-3 (Li et al., 2025) unrolls this supervision over the draft model's own hidden states for the depth of one drafting stage. Loop-aware training starts from that design. Each additional drafting stage in Section 3.2 starts from a chain of $\tilde{h}$ that runs past that depth. Loop-aware training extends the unroll to the chain that one verification covers.

Training scheme. For each training sequence and position t, we sample the number of additional drafting stages $S-1$ uniformly from $\{1, \ldots, N\}$ and run S drafting stages. The gate function decides S during decoding. Training at one length would therefore expose the draft model to a single point in the range it meets. The first drafting stage takes $h_{1:t-1}$ from $\mathcal{M}_t$. Each additional drafting stage takes the chain of final-layer hidden states $\tilde{h}$ that $\mathcal{M}_d$ computed in the preceding drafting stages, as in the loop. The tokens at these positions come from the ground truth. Training therefore reproduces the chain of hidden states that the loop builds and not the draft tokens that accompany it. To simulate the drafting process in Eq. 3, we define $\mathcal{L}^{(s)}$ as the cross-entropy of the s-th drafting stage against the ground-truth tokens $x_{t+(s-1)k+1:t+sk}$. The training loss combines the first and the last drafting stage,

$$\mathcal{L} = \frac{1}{2}\left(\mathcal{L}^{(1)} + \mathcal{L}^{(S)}\right). \tag{4}$$

The first term is the original objective and preserves the first drafting stage. The second term trains the draft model to continue from the previous $\tilde{h}$. The additional drafting stages differ in difficulty. The intermediate drafting stages predict nearer blocks and form the easier part of the loop. The last drafting stage predicts the farthest block from the longest chain of $\tilde{h}$ and forms the hardest part. We place the second loss term only on the last drafting stage. Every training iteration therefore focuses on this hardest part. The intermediate drafting stages do not contribute to the loss and only supply $\tilde{h}$ to the next drafting stage. Their logits are never computed. Loop-aware training does not require

an additional forward pass of the target model, since h is used only for the prefix. The draft model instead runs S drafting stages per training iteration. Loop-aware training leaves the architecture and the parameter count unchanged and applies identically to every draft model on top of its publicly released weights. Tab. A7 ablates N and reports the training time and memory. We use $N = 3$ in our experiments.

4 Experiment

4.1 Experimental Setup

We evaluate DLoop on four draft models, EAGLE-3 (Li et al., 2025), DFlash (Chen et al., 2026), Domino (Huang et al., 2026), and DSpark (Cheng et al., 2026), with Qwen3-4B and Qwen3-8B (Yang et al., 2025) as target models. We also evaluate DLoop with the MTP modules released with Qwen3.5-9B (Team, 2026a) and Gemma-4-12B (Team et al., 2026). The block size k is 16 for DFlash and Domino, 7 for DSpark, 3 for MTP with Qwen3.5-9B, and 6 for MTP with Gemma-4-12B. Loop-aware training starts from the released official weights of each draft model and runs for one epoch. Following prior works (Li et al., 2025; Chen et al., 2026; Huang et al., 2026; Cheng et al., 2026), we construct the training set from the 1.3 million user prompts of PerfectBlend (Xu et al., 2024) paired with responses regenerated by each target model with its recommended sampling parameters. All training hyperparameters follow those released with each method. EAGLE-3 has no officially released weights for these target models. We therefore train EAGLE-3 draft models for three epochs on the same data and use them as the baseline draft model.

We follow the evaluation protocol of DFlash[1], detailed in Appendix A.1, and evaluate on eight benchmarks in three task categories, math (GSM8K (Cobbe et al., 2021), MATH-500 (Lightman et al., 2024), AIME25), code (HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), LiveCodeBench (LCB) (Jain et al., 2025)), and chat (MT-Bench (Zheng et al., 2023), Alpaca (Taori et al., 2023)). We report the acceptance length τ, the average number of tokens accepted per verification, and the wall-clock speedup over autoregressive decoding. All results use greedy decoding ($T = 0$) unless stated otherwise. For tree verification, we apply DDTree (Ringel & Romano, 2026) to DFlash and DARTree (Li et al., 2026) to Domino and DSpark. We follow the maximum number of node budget in the tree that the target model verifies from their papers: 256 for DFlash, 64 for Domino and DSpark. EAGLE-3 uses its own tree verification with the released setting. We sweep the gate threshold g over $\{-0.25, -0.5, -0.75, -1.0\}$, as detailed in the appendix Tab. A5 and select the value with the highest average speedup over the eight benchmarks. We use -0.5 for all settings, except EAGLE-3 with Qwen3-8B, where we use -0.75. All training and evaluation runs use NVIDIA H100 80GB GPUs with bf16 precision.

4.2 Experimental Results

DLoop on recent draft models Table 1 reports the results for the four draft models with Qwen3-4B and Qwen3-8B, and for MTP with Qwen3.5-9B and Gemma-4-12B. Applying DLoop to the publicly released draft models keeps their architecture and block size. DLoop lets the draft model continue drafting after a block instead of verifying each block on its own. This improves both the average acceptance length and the wall-clock speedup for every draft model, target model, and task category. Across these settings, the average acceptance length increases by 12 to 102 percent and the wall-clock speedup by 5 to 41 percent. Among the four draft models with Qwen3-4B and Qwen3-8B, the largest gain is observed for DSpark. DSpark also has the highest full acceptance ratio in Fig. 1 and thus the most blocks after which an additional drafting stage can be accepted. Results at $T = 1$ are reported in Tab. A10.

Ablation of DLoop components To separate the two components of DLoop, we enable the confidence-gated drafting loop and loop-aware training one at a time in Tab. 2. Enabling the loop without loop-aware training increases the average acceptance length for every draft model and target model. For the autoregressive draft models, which already draft each token after the first from their own hidden states, both the acceptance length and the speedup improve for EAGLE-3 with both target models and for MTP with Qwen3.5-9B. In the other settings, the speedup changes little or de-

[1] https://github.com/z-lab/dflash

Table 1: **Decoding speedup over the autoregressive baseline and average acceptance length (τ).** Results are averaged over the benchmarks in each task category and over all eight benchmarks (Avg.). For each draft model, the first row is the publicly released draft model. "+ DLoop" applies one epoch of loop-aware training and enables the confidence-gated drafting loop. Bold marks the better result for each draft model. Per-benchmark results are given in Table A9 in the appendix.

Method	Qwen3-4B				Qwen3-8B			
	MATH	CODE	CHAT	*Avg.*	MATH	CODE	CHAT	*Avg.*
	τ / spd	τ / spd	τ / spd	τ / spd	τ / spd	τ / spd	τ / spd	τ / spd
EAGLE-3 2025	7.83 / 3.85×	7.09 / 3.49×	5.30 / 2.38×	6.92 / 3.35×	7.59 / 3.78×	6.73 / 3.30×	4.98 / 2.26×	6.61 / 3.22×
+ **DLoop**	**12.04 / 4.58**×	**9.43 / 3.83**×	**6.29 / 2.47**×	**9.63 / 3.77**×	**10.45 / 4.31**×	**7.83 / 3.46**×	**5.46 / 2.38**×	**8.22 / 3.51**×
DFlash 2026	7.19 / 5.02×	6.59 / 4.65×	3.74 / 2.27×	6.10 / 4.19×	7.22 / 5.13×	6.56 / 4.45×	3.66 / 2.22×	6.08 / 4.15×
+ **DLoop**	**9.48 / 6.14**×	**6.99 / 4.72**×	**4.12 / 2.38**×	**7.21 / 4.67**×	**9.78 / 6.28**×	**7.21 / 4.70**×	**4.08 / 2.37**×	**7.40 / 4.71**×
Domino 2026	8.85 / 5.63×	6.99 / 4.39×	4.56 / 2.48×	7.08 / 4.38×	9.04 / 5.75×	7.18 / 4.49×	4.54 / 2.51×	7.22 / 4.47×
+ **DLoop**	**10.44 / 5.98**×	**7.41 / 4.54**×	**4.82 / 2.55**×	**7.90 / 4.58**×	**11.14 / 6.31**×	**7.69 / 4.64**×	**4.91 / 2.61**×	**8.29 / 4.76**×
DSpark 2026	6.26 / 4.36×	5.61 / 3.96×	4.09 / 2.62×	5.47 / 3.77×	6.32 / 4.46×	5.81 / 3.96×	4.13 / 2.61×	5.58 / 3.81×
+ **DLoop**	**11.02 / 6.02**×	**8.44 / 4.86**×	**5.17 / 2.71**×	**8.59 / 4.76**×	**11.80 / 6.20**×	**8.84 / 4.83**×	**5.39 / 2.74**×	**9.08 / 4.82**×
	Qwen3.5-9B				**Gemma-4-12B**			
MTP 2026b; 2026	3.54 / 1.74×	3.39 / 1.58×	2.88 / 1.21×	3.32 / 1.55×	5.38 / 3.56×	5.12 / 3.37×	3.71 / 2.28×	4.86 / 3.17×
+ **DLoop**	**8.26 / 2.66**×	**6.92 / 2.26**×	**4.02 / 1.33**×	**6.70 / 2.18**×	**9.06 / 4.33**×	**8.01 / 3.94**×	**4.40 / 2.30**×	**7.50 / 3.68**×

Table 2: **Ablation of the confidence-gated drafting loop and loop-aware training.** Each cell reports the average acceptance length (τ) and speedup (spd) over all eight benchmarks as τ / spd×. "Loop" enables the confidence-gated drafting loop, and "Train" applies one epoch of loop-aware training. The first row is the publicly released draft model, and the last row is DLoop in Table 1. Q4B, Q8B, and Q9B denote Qwen3-4B, Qwen3-8B, and Qwen3.5-9B, and G12B denotes Gemma-4-12B. Bold marks the best result in each column. Per-benchmark results are given in Table A9 in the appendix.

Loop	Train	EAGLE-3		DFlash		Domino		DSpark		MTP	
		Q4B	Q8B	Q4B	Q8B	Q4B	Q8B	Q4B	Q8B	Q9B	G12B
		6.92 / 3.35×	6.61 / 3.22×	6.10 / 4.19×	6.08 / 4.15×	7.08 / 4.38×	7.22 / 4.47×	5.47 / 3.77×	5.58 / 3.81×	3.32 / 1.55×	4.86 / 3.17×
✓		8.04 / 3.52×	7.34 / 3.38×	6.11 / 4.16×	6.20 / 4.17×	7.15 / 4.32×	7.40 / 4.51×	5.77 / 3.79×	5.92 / 3.87×	4.96 / 1.80×	6.06 / 2.90×
	✓	6.99 / 3.37×	6.60 / 3.15×	6.38 / 4.23×	6.48 / 4.33×	7.06 / 4.38×	7.23 / 4.52×	5.41 / 3.73×	5.50 / 3.77×	3.37 / 1.60×	4.95 / 3.22×
✓	✓	**9.63 / 3.77**×	**8.22 / 3.51**×	**7.21 / 4.67**×	**7.40 / 4.71**×	**7.90 / 4.58**×	**8.29 / 4.76**×	**8.59 / 4.76**×	**9.08 / 4.82**×	**6.70 / 2.18**×	**7.50 / 3.68**×

creases, since the time added by the additional drafting stages largely offsets the gain in acceptance length. Loop-aware training alone changes both metrics by less than 7 percent, leaving them close to those of the publicly released draft models. Combined with the loop, however, it improves both metrics in every setting, and the gain over the publicly released draft model exceeds the sum of the gains from the loop alone and the training alone. Loop-aware training thus prepares the draft model for the loop rather than improving each drafting stage on its own. Detailed design choices for the gate function and loop-aware training are provided in Appendix A.7.

Comparison with adaptive draft length methods. The adaptive draft length methods in Section 2 also decide during decoding how many draft tokens precede a verification. BiLD, SVIP, AdaEDL, LLMCad, and DDD stop drafting once a statistic of the draft distribution, such as the top-1 probability or the entropy, crosses a threshold. SpecDec++ and DISCO instead train a head that decides after each draft token whether to stop, whereas AdaEAGLE trains a predictor that sets the draft length in advance. PACER trains a pre-verification layer and drafts another block while the acceptance this layer predicts remains above a threshold. The confidence scheduler of DSpark chooses how many draft tokens of the full block to verify, using a confidence head trained together with DSpark. BiLD, SVIP, AdaEDL, LLMCad, SpecDec++, DISCO, and PACER were proposed for a separate small autoregressive LLM, whose draft length can be varied freely because it is a standalone language model. In contrast, EAGLE-3 and DSpark are trained for a *fixed draft length*, and drafting beyond this length quickly loses efficiency. We apply these methods to EAGLE-3 and DSpark as detailed in Appendix A.8. Because the parallel draft model DSpark computes all k draft tokens of a drafting stage in a single forward pass, cutting its draft shortens only the verification. On DSpark, PACER and DDD instead decide at the end of each drafting stage whether another drafting stage follows. We train the heads of SpecDec++, DISCO, AdaEAGLE, and PACER for one epoch on the same

Table 3: **Comparison with adaptive draft length methods.** All methods are applied to EAGLE-3, an autoregressive draft model, and DSpark, a parallel draft model, with Qwen3-4B. Each cell reports the average acceptance length (τ) and speedup (spd) as τ / spd×, averaged over the benchmarks in each task category and over all eight benchmarks (Avg.). Cells without results are methods that are not applicable to the draft model, as discussed in Section 4.2. The first row is the publicly released draft model, and "+ DLoop" is the same as in Table 1. Bold marks the best result in each column, and underline marks results higher than the draft model.

	EAGLE-3				DSpark			
Method	MATH	CODE	CHAT	*Avg.*	MATH	CODE	CHAT	*Avg.*
Draft model	7.83 / 3.85×	7.09 / 3.49×	5.30 / 2.38×	6.92 / 3.35×	6.26 / 4.36×	5.61 / 3.96×	4.09 / 2.62×	5.47 / 3.77×
+ BiLD (Kim et al., 2023)	8.70 / 4.01×	7.63 / 3.49×	5.31 / 2.31×	7.45 / 3.39×	6.26 / 4.31×	5.61 / 3.93×	4.09 / 2.60×	5.47 / 3.74×
+ SVIP (Zhang et al., 2025)	7.73 / 3.86×	7.06 / 3.51×	5.10 / 2.40×	6.82 / 3.36×	6.08 / 4.17×	5.56 / 3.90×	3.94 / 2.47×	5.35 / 3.65×
+ AdaEDL (Agrawal et al., 2024)	7.83 / 3.84×	7.09 / 3.46×	5.20 / 2.37×	6.90 / 3.33×	6.25 / 4.30×	5.61 / 3.93×	4.08 / 2.57×	5.47 / 3.73×
+ LLMCad (Xu et al., 2023)	9.18 / 4.10×	7.40 / 3.53×	5.09 / **2.51**×	7.49 / 3.49×	6.11 / 4.18×	5.45 / 3.79×	3.69 / 2.29×	5.26 / 3.56×
+ SpecDec++ (Huang et al., 2025)	8.85 / 3.87×	7.44 / 3.40×	5.04 / 2.43×	7.37 / 3.33×	6.27 / 4.32×	5.60 / 3.91×	4.05 / 2.56×	5.46 / 3.73×
+ DISCO (Mamou et al., 2024)	7.71 / 3.67×	7.04 / 3.35×	5.10 / 2.38×	6.81 / 3.23×	6.27 / 4.31×	5.61 / 3.88×	4.09 / 2.57×	5.48 / 3.71×
+ PACER (Zhang et al., 2026)	9.08 / 3.84×	7.66 / 3.28×	5.32 / 2.35×	7.61 / 3.26×	6.85 / 4.22×	5.92 / 3.75×	4.19 / 2.53×	5.83 / 3.62×
+ AdaEAGLE (Zhang et al., 2024b)	7.71 / 3.86×	6.94 / 3.51×	4.83 / 2.46×	6.70 / 3.38×	6.23 / 4.30×	5.55 / 3.87×	3.79 / 2.37×	5.37 / 3.66×
+ DDD (Brown et al., 2024)	8.76 / 4.20×	7.71 / 3.73×	5.38 / 2.41×	7.52 / 3.58×	6.71 / 4.39×	5.89 / 3.94×	4.19 / 2.59×	5.77 / 3.77×
+ DSpark's confidence scheduler	*specific to DSpark*				6.26 / 4.38×	5.61 / 3.87×	4.09 / 2.63×	5.47 / 3.75×
+ **DLoop**	**12.04 / 4.58**×	**9.43 / 3.83**×	**6.29** / 2.47×	**9.63 / 3.77**×	**11.02 / 6.02**×	**8.44 / 4.86**×	**5.17 / 2.71**×	**8.59 / 4.76**×

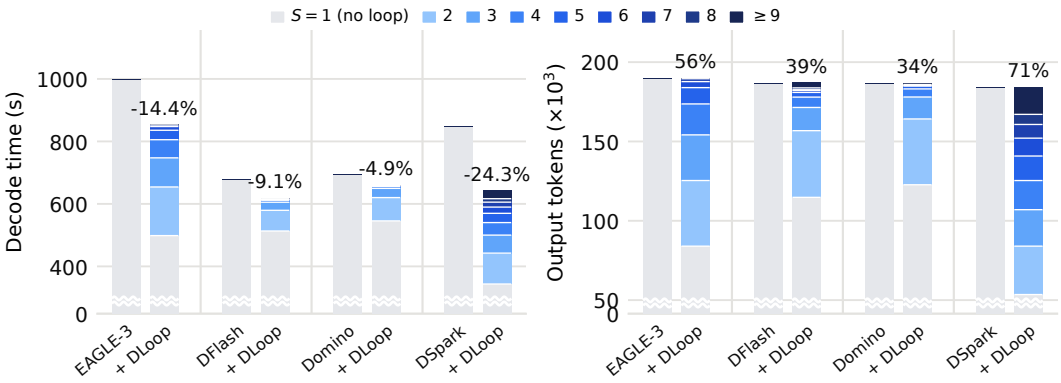


Figure 3: **Decode time and output tokens by the number of drafting stages.** We break down the total decode time (left) and output tokens (right) by the number of drafting stages S in each loop, summed over the math benchmarks with Qwen3-4B. The percentages above the left bars show the reduction in total decode time from DLoop, while those above the right bars show the fraction of output tokens generated in loops with $S > 1$.

data as loop-aware training. Tab. 3 shows that on DSpark no method exceeds the speedup of the draft model, and on EAGLE-3 the seven methods above raise it by at most 0.14×, as their stopping decisions add computation to the drafting loop. With loop-aware training for the additional drafting stages, DLoop gives the highest average acceptance length and speedup on both draft models.

Where the speedup comes from. Fig. 3 summarizes the total decode time and the output tokens by the number of drafting stages S over the math benchmarks. DLoop reduces the total decode time for all four draft models by 4.9–24.3%. Among the generated tokens, 34–71% are produced in loops with $S > 1$, where multiple drafting stages are covered by a single verification rather than one verification per drafting stage. Table 4 shows what one verification yields by S. The time per loop increases with S, as more drafting stages are performed before each verification. However, the relative increase in τ with S is larger than that in the time per loop, resulting in a monotonic decrease in the time per output token from 4.60–6.42 ms at $S = 1$ to 0.68–2.97 ms at $S \geq 5$. Overall, DLoop reduces the number of verifications by 7.6–34.1%. Since each verification requires a forward pass of the target model and dominates the cost of a loop, the reduction in verifications outweighs the additional drafting stages. DLoop therefore obtains its speedup by spending additional draft-model forward passes to reduce the number of target-model forward passes.

DLoop with tree verification. Table 5 shows that DLoop further raises the acceptance length and the speedup when applied on top of tree verification. Tree verification broadens the draft with

Table 4: **Time and output tokens per loop by the number of drafting stages.** Results on the math benchmarks with Qwen3-4B. Loops is the number of loops (i.e., verifications), ms/L the average time per loop, τ/L the acceptance length per loop, and ms/tok the average time per output token. No loop denotes the draft model without DLoop, and All S aggregates all loops of DLoop.

	EAGLE-3				**DFlash**				**Domino**				**DSpark**			
	Loops	ms/L	τL	ms/tok	Loops	ms/L	τ/L	ms/tok	Loops	ms/L	τ/L	ms/tok	Loops	ms/L	τ/L	ms/tok
No loop	24,361	40.91	7.88	5.259	24,839	27.36	8.00	3.643	23,061	30.11	8.83	3.717	30,471	27.84	6.17	4.614
S=1	11,974	40.91	7.14	5.934	19,702	27.36	6.28	4.625	18,719	30.11	7.06	4.598	12,481	27.84	4.65	6.423
S=2	2,828	54.10	14.23	3.763	2,068	32.53	20.60	1.576	1,977	37.69	20.96	1.798	3,091	32.33	10.09	3.238
S=3	1,361	67.29	20.88	3.239	465	37.70	31.91	1.180	435	45.27	33.01	1.425	1,585	36.81	14.69	2.510
S=4	709	80.48	27.33	2.977	148	42.87	44.34	0.958	116	52.86	50.58	1.203	982	41.30	19.27	2.183
$S\geq5$	473	99.44	32.80	2.968	114	55.70	73.01	0.681	61	67.90	66.91	1.060	1,944	53.13	30.80	1.735
All S	**17,345**	**48.34**	**12.04**	**4.502**	**22,497**	**28.30**	**9.48**	**3.311**	**21,308**	**31.35**	**10.44**	**3.535**	**20,083**	**32.35**	**11.02**	**3.493**

Table 5: **Results with tree verification.** Complementary to tree verification, DLoop further raises the average acceptance length (τ) and speedup (spd) of every draft model. For each draft model, the first row is the publicly released draft model. "+ Tree" applies tree verification to it. "+ Tree + DLoop" further applies DLoop. Bold marks the best result for each draft model. Per-benchmark results are given in Table A9 in the appendix.

Method	**Qwen3-4B**				**Qwen3-8B**			
	MATH	CODE	CHAT	*Avg.*	MATH	CODE	CHAT	*Avg.*
	τ / spd	τ / spd	τ / spd	τ / spd	τ / spd	τ / spd	τ / spd	τ / spd
DFlash 2026	7.19 / 5.02×	6.59 / 4.65×	3.74 / 2.27×	6.10 / 4.19×	7.22 / 5.13×	6.56 / 4.45×	3.66 / 2.22×	6.08 / 4.15×
+ DDTree 2026	9.66 / 6.73×	9.23 / 6.33×	5.67 / 3.56×	8.50 / 5.79×	9.66 / 6.82×	9.25 / 6.44×	5.56 / 3.57×	8.48 / 5.87×
+ DDTree + **DLoop**	**13.27 / 8.11×**	**10.20 / 6.62×**	**6.27 / 3.81×**	**10.37 / 6.48×**	**12.97 / 8.35×**	**10.18 / 6.78×**	**6.12 / 3.79×**	**10.22 / 6.62×**
Domino 2026	8.85 / 5.63×	6.99 / 4.39×	4.56 / 2.48×	7.08 / 4.38×	9.04 / 5.75×	7.18 / 4.49×	4.54 / 2.51×	7.22 / 4.47×
+ DARTree 2026	11.78 / 6.15×	9.83 / 5.26×	6.58 / 3.14×	9.75 / 5.06×	11.86 / 6.31×	10.06 / 5.29×	6.59 / 3.10×	9.87 / 5.13×
+ DARTree + **DLoop**	**14.22 / 6.83×**	**10.81 / 5.43×**	**7.12 / 3.18×**	**11.17 / 5.40×**	**15.08 / 6.99×**	**11.38 / 5.59×**	**7.21 / 3.20×**	**11.73 / 5.52×**
DSpark 2026	6.26 / 4.36×	5.61 / 3.96×	4.09 / 2.62×	5.47 / 3.77×	6.32 / 4.46×	5.81 / 3.96×	4.13 / 2.61×	5.58 / 3.81×
+ DARTree 2026	6.94 / 4.61×	6.78 / 4.47×	5.23 / 3.25×	6.45 / 4.22×	7.08 / 4.58×	6.93 / 4.40×	5.42 / 3.35×	6.61 / 4.21×
+ DARTree + **DLoop**	**12.60 / 6.10×**	**10.08 / 5.40×**	**6.52 / 3.35×**	**10.14 / 5.15×**	**12.31 / 6.36×**	**10.16 / 5.42×**	**6.72 / 3.45×**	**10.11 / 5.28×**

branches that hold alternative tokens at the same position, whereas DLoop lengthens it with additional drafting stages. On a tree, DLoop extends the highest-scoring path out of tree branches. The gate function scores this path as in Section 3.2. The draft tokens of each additional drafting stage form another tree rooted at the end of this path for DDTree and a single chain for DARTree. The target model verifies the tree and the added draft tokens together in one forward pass. Over tree verification alone, DLoop raises the average speedup of DFlash with DDTree by 12 to 13 percent. With DARTree, the gain is 7 to 8 percent for Domino and 22 to 25 percent for DSpark. Since EAGLE-3 uses its own tree verification, its results in Table 1 already show this gain.

5 Conclusion

We introduced DLoop, a looped form of speculative decoding in which a single verification covers multiple drafting stages. The confidence-gated drafting loop repeats the drafting stage while the draft model remains confident. Loop-aware training keeps the draft model reliable in the additional drafting stages by exposing it to its own hidden states for unverified draft tokens. DLoop leaves the architecture of the draft model and the acceptance rule unchanged. It changes only how many drafting stages precede a verification. Across EAGLE-3, DFlash, Domino, DSpark, and multi-token prediction modules, DLoop improves the wall-clock speedup by 5 to 41 percent while preserving lossless decoding. It obtains this speedup by spending additional draft-model forward passes to reduce the number of target-model forward passes. Adaptive draft length methods raise the speedup only for autoregressive draft models, whereas DLoop raises it for parallel draft models as well. The ablation shows that loop-aware training prepares the draft model for the loop rather than improving each drafting stage on its own. Applied on top of tree verification, DLoop further raises the acceptance length and the speedup. These results show that the number of drafting stages before a verification need not be fixed by the schedule and can be decided during decoding.

AI USE STATEMENT

We used generative AI tools to aid or polish the writing of this paper. We also used them to assist in scheduling the training and evaluation experiments on a GPU cloud. In implementing the methods, we used them to a limited extent, mainly for debugging and error handling. We reviewed all text and code written with these tools. For synthetic data sets, we regenerated only the responses to the user prompts of PerfectBlend (Xu et al., 2024) with each target model, as described in Section 4.1. We did not use generative AI tools to develop theoretical models or conceptual frameworks or to propose or refine hypotheses. These tools did not design or provide feedback on the research methodology or experiments and did not interpret the results. They also did not formulate mathematical claims, provide critical ingredients for proving them, or assist in writing proofs. We did not use them to assist with translation, to clean and reformat data sets, or to support qualitative and thematic data analysis. We take responsibility for all text, claims, and artifacts of this work, including those produced with generative AI tools.

ETHICS STATEMENT

This work does not raise questions regarding the ICLR Code of Ethics. DLoop accelerates decoding and preserves the output distribution of the target model.

REPRODUCIBILITY STATEMENT

Section 4.1 describes the draft models, the target models, the training data, the benchmarks, and the hyperparameters. The appendix details the evaluation protocol and the comparison with adaptive draft length methods. Upon publication, we will release all code needed to reproduce the experiments in this paper.

REFERENCES

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Sudhanshu Agrawal, Wonseok Jeon, and Mingu Lee. Adaedl: Early draft stopping for speculative decoding of large language models via an entropy-based lower bound on token acceptance probability. *arXiv preprint arXiv:2410.18351*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Oscar Brown, Zhengjie Wang, Andrea Do, Nikhil Mathew, and Cheng Yu. Dynamic depth decoding: Faster speculative decoding for llms. *arXiv preprint arXiv:2409.00142*, 2024.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Forty-first International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=PEpbUobfJv.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Xin Cheng, Xingkai Yu, Chenze Shao, Jiashi Li, Yunfan Xiong, Yi Qian, Jiaqi Zhu, Shirong Ma, Xiaokang Zhang, Jiasheng Ye, et al. Dspark: Confidence-scheduled speculative decoding with semi-autoregressive generation. *arXiv preprint arXiv:2607.05147*, 2026.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Geonmo Gu, Byeongho Heo, HeeJae Jun, Yoohoon Kang, Sangmin Lee, Sangdoo Yun, and Dongyoon Han. Verification-aware training for speculative decoding. *arXiv preprint arXiv:2608.30135*, 2026.

Jianuo Huang, Yaojie Zhang, Qituan Zhang, Hao Lin, Hanlin Xu, and Linfeng Zhang. Domino: Decoupling causal modeling from autoregressive drafting in speculative decoding. *arXiv preprint arXiv:2605.29707*, 2026.

Kaixuan Huang, Xudong Guo, and Mengdi Wang. Specdec++: Boosting speculative decoding via adaptive candidate lengths. In *Second Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=Y131N9fUbU.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=chfJJYC3iL.

Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W Mahoney, Amir Gholami, and Kurt Keutzer. Speculative decoding with big little decoder. *Advances in Neural Information Processing Systems*, 36:39236–39256, 2023.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International conference on machine learning*, pp. 19274–19286. PMLR, 2023.

Tianyi Li, Yaxin Luo, Xinyi Shang, and Zhiqiang Shen. Dartree: Speculative diffusion decoding with autoregressive draft trees. *arXiv preprint arXiv:2608.13524*, 2026.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Forty-first International Conference on Machine Learning*, 2024a. URL https://openreview.net/forum?id=1NdN7eXyb4.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pp. 7421–7432, 2024b.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=4exx1hUffq.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, volume 2024, pp. 39578–39601, 2024.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024a.

Xiaoxuan Liu, Jongseok Park, Langxiang Hu, Woosuk Kwon, Zhuohan Li, Chen Zhang, Kuntai Du, Xiangxi Mo, Kaichao You, Alvin Cheung, et al. Turbospec: Closed-loop speculation control system for optimizing llm serving goodput. *arXiv preprint arXiv:2406.14066*, 2024b.

Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe Berchansky, Nadav Timor, Moshe Wasserblat, and Roy Schwartz. Dynamic speculation lookahead accelerates speculative decoding of large language models. *arXiv preprint arXiv:2405.04304*, 2024.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pp. 932–949, 2024.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of machine learning and systems*, 5:606–624, 2023.

Liran Ringel and Yaniv Romano. Accelerating speculative decoding with block diffusion draft trees. In *Third Conference on Language Modeling*, 2026. URL https://openreview.net/forum?id=2ivmZcTz07.

Weijie Shi, Qiang Xu, Fan Deng, Yaguang Wu, Jiarun Liu, Yehong Xu, Hao Chen, Jia Zhu, Jiajie Xu, Xiangjun Huang, et al. Specblock: Block-iterative speculative decoding with dynamic tree drafting. *arXiv preprint arXiv:2605.07243*, 2026.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31, 2018.

Qidong Su, Christina Giannoula, and Gennady Pekhimenko. The synergy of speculative decoding and batching in serving large language models. *arXiv preprint arXiv:2310.18813*, 2023.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models. https://crfm. stanford. edu/2023/03/13/alpaca. html*, 3(6):7, 2023.

Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

Gemma Team, Sherif El Abd, Vaibhav Aggarwal, Robin Algayres, Alek Andreev, Olivier Bachem, Ian Ballantyne, Cormac Brick, Victor Cărbune, Michelle Casbon, et al. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026.

Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026a. URL https://qwen.ai/blog?id=qwen3.5.

Qwen Team. Qwen3. 5-omni technical report. *arXiv preprint arXiv:2604.15804*, 2026b.

Daliang Xu, Wangsong Yin, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. Llmcad: Fast and scalable on-device large language model inference. *arXiv preprint arXiv:2309.04255*, 2023.

Tengyu Xu, Eryk Helenowski, Karthik Abinav Sankararaman, Di Jin, Kaiyan Peng, Eric Han, Shaoliang Nie, Chen Zhu, Hejia Zhang, Wenxuan Zhou, et al. The perfect blend: Redefining rlhf with mixture of judges. *arXiv preprint arXiv:2409.20370*, 2024.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft& verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11263–11282, 2024a.

Situo Zhang, Hankun Wang, Da Ma, Zichen Zhu, Lu Chen, Kunyao Lan, and Kai Yu. Adaeagle: Optimizing speculative decoding via explicit modeling of adaptive draft structures. *arXiv preprint arXiv:2412.18910*, 2024b.

Situo Zhang, Yifan Zhang, Zichen Zhu, Hankun Wang, Da Ma, Danyang Zhang, Lu Chen, and Kai Yu. Pacer: Blockwise pre-verification for speculative decoding with adaptive length. *IEEE Transactions on Audio, Speech and Language Processing*, 2026.

Ziyin Zhang, Jiahao Xu, Tian Liang, Xingyu Chen, Zhiwei He, Rui Wang, and Zhaopeng Tu. Draft model knows when to stop: Self-verification speculative decoding for long-form generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 16696–16708, 2025.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.

A APPENDIX

- Appendix A.1 describes the evaluation protocol, including the benchmark sources, the sample counts, and the prompt templates.
- Appendix A.2 reports the results on each benchmark with greedy decoding and with sampling at temperature $T = 1$.
- Appendix A.3 evaluates the decoding throughput of DLoop in SGLang under concurrent serving.
- Appendix A.4 analyzes how DLoop changes the total numbers of drafting stages and verifications.
- Appendix A.5 measures the time and the peak GPU memory of one loop by the number of drafting stages.
- Appendix A.6 presents representative generations of the confidence-gated drafting loop.
- Appendix A.7 compares design choices for the gate threshold, the gate function, and loop-aware training.
- Appendix A.8 details how we apply the adaptive draft length methods to EAGLE-3 and DSpark.

A.1 EVALUATION PROTOCOL

The eight benchmarks and their sample counts follow the official DFlash evaluation code. Table A1 lists the source and the number of evaluated samples for each benchmark. Each subset of samples consists of the first samples upto the number of samples after shuffling the benchmark with seed 0. Table A8 lists the prompt templates of the DFlash evaluation code. MBPP and MT-Bench use the text of each sample without a template. We place each prompt in a user message and apply the chat template of the target model. We do not add a system prompt and disable thinking mode. The instruction at the start of the LiveCodeBench (LCB) template is therefore part of the user message. Every result on the eight benchmarks follows this protocol.

Table A1: **Number of evaluated samples per benchmark.** Size is the number of samples in the source dataset on Hugging Face.

Benchmark	Source	Size	Samples
MATH			
GSM8K (Cobbe et al., 2021)	`openai/gsm8k`	1,319	128
MATH-500 (Lightman et al., 2024)	`HuggingFaceH4/MATH-500`	500	128
AIME25	`MathArena/aime_2025`	30	30
CODE			
HumanEval (Chen et al., 2021)	`openai/openai_humaneval`	164	164
MBPP (Austin et al., 2021)	`google-research-datasets/mbpp`	257	128
LCB (Jain et al., 2025)	`livecodebench/code_generation_lite`	1,055	128
CHAT			
MT-Bench (Zheng et al., 2023)	`HuggingFaceH4/mt_bench_prompts`	80	80
Alpaca (Taori et al., 2023)	`tatsu-lab/alpaca`	52,002	128

A.2 PER-BENCHMARK RESULTS

Table A9 reports the acceptance length and the speedup of every setting in Tables 1, 2, and 5 on each benchmark. DLoop raises the acceptance length on every benchmark for every draft model and target model. The gain is largest on math and smallest on chat. The full acceptance ratio in Fig. 1 follows the same order. Table A10 reports the same settings with sampling at temperature $T = 1$. At $T = 1$, DLoop also raises the average acceptance length.

A.3 SERVING THROUGHPUT UNDER CONCURRENCY.

Table A2 evaluates DLoop with both DFlash and DSpark in SGLang under concurrent serving with Qwen3-8B as the target model. DLoop improves throughput substantially at low concurrency. On MATH-500 at $c = 1$, the speedup increases from 5.81× to 6.23× with DFlash and from 4.45× to 4.94× with DSpark, with similar gains on HumanEval. The gain generally becomes smaller as concurrency increases, but DLoop remains beneficial through $c = 8$ with DFlash and through $c = 16$

with DSpark on both benchmarks. At $c = 32$, DFlash+DLoop slightly underperforms DFlash, whereas DSpark+DLoop continues to improve throughput on MATH-500 and remains within 1% of DSpark on HumanEval. This attenuation is consistent with the known interaction between speculative decoding and batching, in which additional speculative work consumes increasingly scarce batch capacity as the target model becomes saturated, reducing the optimal speculation depth at high load (Su et al., 2023; Liu et al., 2024b; Cheng et al., 2026). DLoop further chooses the number of drafting stages independently for each sample, yielding ragged draft-side execution whose efficient batching requires either padding, which wastes computation, or dynamic regrouping, which introduces additional scheduling overhead. Such sample-dependent execution therefore leaves room for serving and kernel optimizations that better support adaptive loop depths. Overall, DLoop retains substantial throughput gains at low-to-moderate concurrency, while its advantage diminishes as serving becomes increasingly batch-bound.

Table A2: **SGLang decoding throughput with DFlash, DSpark, and DLoop under different concurrency levels.** We report throughput in tokens per second (tok/s), followed by speedup over the autoregressive (AR) baseline. **Bold** indicates the better result within each model pair.

Method	$c = 1$	$c = 2$	$c = 4$	$c = 8$	$c = 16$	$c = 32$
MATH-500						
AR	156 / 1.00×	303 / 1.00×	593 / 1.00×	1,138 / 1.00×	2,132 / 1.00×	3,763 / 1.00×
DFlash	907 / 5.81×	1,750 / 5.78×	3,252 / 5.48×	6,019 / 5.29×	**9,484 / 4.45**×	**11,810 / 3.14**×
+ **DLoop**	**972 / 6.23**×	**1,833 / 6.05**×	**3,362 / 5.67**×	**6,134 / 5.39**×	9,189 / 4.31×	11,139 / 2.96×
DSpark	694 / 4.45×	1,244 / 4.11×	2,517 / 4.24×	4,538 / 3.99×	8,117 / 3.81×	12,265 / 3.26×
+ **DLoop**	**770 / 4.94**×	**1,480 / 4.88**×	**2,692 / 4.54**×	**4,874 / 4.28**×	**8,266 / 3.88**×	**12,820 / 3.41**×
HumanEval						
AR	156 / 1.00×	301 / 1.00×	595 / 1.00×	1,149 / 1.00×	2,154 / 1.00×	3,762 / 1.00×
DFlash	717 / 4.60×	1,357 / 4.51×	2,552 / 4.29×	4,673 / 4.07×	7,489 / 3.48×	**9,451 / 2.51**×
+ **DLoop**	**779 / 4.98**×	**1,454 / 4.83**×	**2,648 / 4.45**×	**4,791 / 4.17**×	**7,668 / 3.56**×	9,104 / 2.42×
DSpark	641 / 4.11×	1,228 / 4.07×	2,243 / 3.77×	4,232 / 3.68×	7,234 / 3.36×	**10,926 / 2.90**×
+ **DLoop**	**714 / 4.58**×	**1,326 / 4.41**×	**2,451 / 4.12**×	**4,317 / 3.76**×	**7,297 / 3.39**×	10,825 / 2.88×

A.4 Change in the number of drafting stages and verifications

Table A3 shows how DLoop changes the total numbers of drafting stages and verifications over the eight benchmarks. Without tree verification, DLoop increases the number of drafting stages by 3.5–24.5% and reduces the number of verifications by 4.3–22.7% across the four draft models. On top of tree verification, DLoop increases the number of drafting stages by 4.4–37.9% and reduces the number of verifications by 4.6–26.9%. In both settings, the gains in acceptance length and speedup rank the draft models in the same order as the reduction in verifications. The reduction is largest for DSpark and smallest for Domino. For DSpark and for DFlash with tree verification, the added drafting stages outnumber the removed verifications. The speedup still increases by 10.2–27.6% in these settings. Table A4 shows that each additional drafting stage takes less time than one verification for every draft model and every S. The reduction in verifications therefore outweighs the additional drafting stages.

A.5 Cost of a loop by the number of drafting stages

Table A4 reports the time and the peak GPU memory of one loop for S from 1 to 4. At $S = 1$, the loop reduces to standard speculative decoding. For every draft model, the drafting time increases almost linearly with S. The verification time remains within 21.9–25.0 ms, although the verification covers more draft tokens as S grows. One drafting stage costs less than one verification, even for EAGLE-3. The peak GPU memory increases by less than 0.1 GB from $S = 1$ to $S = 4$.

A.6 Examples of the drafting loop.

Fig. A1 shows representative generations with DSpark to illustrate how the confidence-gated drafting loop behaves within an output. The number of drafting stages S varies across loops. Some loops stop after the first drafting stage ($S = 1$), while confident loops continue for two, three, or four drafting stages before a single verification, as indicated by the highlighted spans. The examples show

Table A3: **Number of drafting stages and verifications.** The baseline performs one drafting stage per verification. DLoop trades additional drafting stages for fewer verifications. The counts are measured on Qwen3-4B over the eight benchmarks, and the baseline is the loop-aware tuned draft model. The changes in acceptance length (τ) and speedup (spd) are computed from the corresponding rows of Table 1.

	Baseline		**+ DLoop**		**Change**			
Setup	Drafting	Verification	Drafting	Verification	τ	Speedup	Drafting	Verification
EAGLE-3	70,311	70,311	78,119	57,594	+37.8%	+11.9%	+11.1%	−18.1%
DFlash	87,172	87,172	90,601	82,611	+13.0%	+10.4%	+3.9%	−5.2%
+ Tree	58,924	58,924	65,162	53,721	+15.5%	+10.2%	+10.6%	−8.8%
Domino	78,415	78,415	81,192	75,023	+11.9%	+4.6%	+3.5%	−4.3%
+ Tree	51,689	51,689	53,989	49,314	+8.1%	+7.1%	+4.4%	−4.6%
DSpark	91,851	91,851	114,391	70,978	+58.8%	+27.6%	+24.5%	−22.7%
+ Tree	75,753	75,753	104,460	55,349	+47.8%	+25.3%	+37.9%	−26.9%

Table A4: **Latency and peak GPU memory by the number of drafting stages.** Each loop runs a fixed number S of drafting stages and one verification. Measured with Qwen3-4B (Yang et al., 2025) on a single H100 GPU in bf16 with a 1,024-token prefix. Draft is the total time of the S drafting stages in ms. Verify is the time of one verification over all draft tokens in ms. The gate function takes at most 0.33 ms per loop and is excluded.

	EAGLE-3			**DFlash**			**Domino**			**DSpark**		
S	Draft	Verify	Mem.	Draft	Verify	Mem.	Draft	Verify	Mem.	Draft	Verify	Mem.
1	12.10	23.09	9.08	3.66	21.99	8.91	6.00	22.75	9.01	3.90	22.45	9.06
2	23.13	22.63	9.10	7.23	22.46	8.92	11.70	22.52	9.02	7.58	23.24	9.06
3	34.72	22.78	9.13	10.83	23.30	8.92	17.69	23.18	9.02	12.41	23.32	9.06
4	47.32	23.37	9.16	13.96	23.56	8.93	23.81	23.18	9.03	15.04	24.32	9.06

that DLoop does not use a fixed number of drafting stages throughout generation, but selectively extends drafting only in loops where the gate permits it. This allows multiple consecutive blocks to be covered by one target-model verification and reduces the end-to-end decoding latency compared with the same DSpark model without the loop.

A.7 Design Choices for Gate Function and Loop-Aware Training

A.7.1 Gate threshold.

Table A5 reports the average acceptance length and speedup of DLoop for each gate threshold g.

Table A5: **Gate threshold sweep.** Each cell reports acceptance length and speedup (τ / spd×) of DLoop averaged over the eight benchmarks at $T = 0$.

g	**EAGLE-3**		**DFlash**		**Domino**		**DSpark**		**MTP**	
	Q4B	Q8B	Q4B	Q8B	Q4B	Q8B	Q4B	Q8B	Q9B	G12B
−0.25	9.01 / 3.71×	7.59 / 3.44×	7.21 / 4.60×	7.39 / 4.66×	7.83 / 4.51×	8.19 / 4.74×	8.51 / 4.70×	8.54 / 4.74×	5.41 / 2.06×	6.35 / 3.58×
−0.5	9.63 / **3.77**×	7.87 / 3.46×	7.21 / **4.67**×	7.40 / **4.71**×	7.90 / **4.58**×	8.29 / **4.76**×	8.59 / **4.76**×	9.08 / **4.82**×	6.70 / **2.18**×	7.50 / **3.68**×
−0.75	10.12 / 3.75×	8.22 / **3.51**×	7.29 / 4.52×	7.50 / 4.49×	8.03 / 4.51×	8.52 / 4.69×	9.14 / 4.52×	9.40 / 4.69×	7.10 / 2.14×	7.82 / 3.65×
−1.0	10.77 / 3.55×	8.51 / 3.42×	7.35 / 4.48×	7.58 / 4.41×	8.15 / 4.48×	8.62 / 4.59×	9.33 / 4.22×	9.69 / 4.40×	7.46 / 2.10×	8.23 / 3.41×

A.7.2 Gate function.

Table A6 compares the log-probability score used by DLoop with alternative confidence scores under the same drafting budget. The log probability gives the highest AUROC and precision for all four draft models, and also gives the highest speedup among the practical gate functions. The learned per-token head attains slightly longer acceptance length for DFlash and Domino, but this does not trans-

User Query

```
Benjamin went to a bookstore and purchased a variety
of books. He bought 5 copies of a sci-fi novel, each
priced at $20, 3 copies of a history book priced at
$30 each, and 2 copies of a philosophy book for $45
each.
What was the total cost of his purchases?
```

Output

```
Let's calculate the total cost of each type of book
and then add them together.
1. **Sci-fi novels**:
   - 5 copies at $20 each:
     $ 5 \times 20 =  100 $ dollars
2. **History books**:
   - 3 copies at $30 each:
     $ 3 \times 30 = 90 $ dollars
3. **Philosophy books**:
   - 2 copies at $45 each:
     $ 2 \times 45 = 90 $ dollars
Now, add them all together:
$  100 + 90 + 90 = 280 $
### **Total cost of Benjamin's purchases:
$280**.<|im_end|>
```

User Query

```
Merge two lists of length 6 into a single list.
Input:
List 1: 1, 3, 5, 7, 9, 11
List 2: 2, 4, 6, 8, 10, 12
```

Output

```
To merge two lists of equal length (6 elements each) into
a single list, you can simply concatenate them. Here's
how you can do it:
### Input:
- **List 1**: `[1, 3, 5, 7, 9, 11]`
- **List 2**: `[2, 4, 6, 8, 10, 12]`
### Merged List:
```python
merged_list = List1 + List2
```
Result:
```python
[1, 3, 5, 7, 9, 11, 2, 4, 6, 8 , 10, 12]
```
Let me know if you want to interleave the elements or
perform any other operation!<|im_end|>
```

Figure A1: **Examples of the confidence-gated drafting loop.** We show representative generations illustrating how the number of drafting stages S varies across loops. Highlighted spans indicate output tokens generated in loops with $S = 2$, $S = 3$, and $S = 4$ drafting stages, respectively; unhighlighted spans correspond to loops with $S = 1$. In each loop, DLoop performs the indicated number of drafting stages before a single target-model verification.

late into higher speedup. The learned heads require additional parameters and a separate training stage, yet remain below the parameter-free scores in AUROC and precision. Entropy is the strongest parameter-free alternative, but is consistently below the log probability in AUROC and precision. Entropy measures uncertainty over the entire draft distribution, whereas the log probability directly measures the probability assigned to the draft tokens that will be checked by the acceptance rule. Detailed threshold-sweep results for each gate function are provided in the appendix.

Table A6: **Confidence scores for the gate with Qwen3-4B.** A block is positive if all of its draft tokens are accepted; for EAGLE-3, a block is the highest-scoring root-to-leaf path. AUROC measures ranking quality, while precision is the fraction of continued blocks that are positive, evaluated at the same continuation rate as our log-probability gate. Acceptance length (τ) and speedup (spd) are averaged over eight benchmarks. Tree verification is disabled except for EAGLE-3, and Oracle continues only on positive blocks. **Bold** indicates the best non-oracle result for each metric

	EAGLE-3				**DFlash**				**Domino**				**DSpark**			
Gate	AUROC	Prec.	τ	spd	AUROC	Prec.	τ	spd	AUROC	Prec.	τ	spd	AUROC	Prec.	τ	spd
No loop	–	–	6.99	3.37	–	–	6.38	4.23	–	–	7.06	4.38	–	–	5.41	3.73
DSpark head	–	–	–	–	–	–	–	–	–	–	–	–	0.909	75.0	8.47	4.68
Learned head (per token)	0.810	58.3	9.31	3.62	0.975	69.9	**7.24**	4.57	0.927	63.3	**7.94**	4.45	0.907	74.7	8.56	4.65
Learned head (pooled)	0.799	56.6	8.81	3.58	0.974	69.1	7.17	4.57	0.924	61.0	7.58	4.43	0.909	75.3	8.13	4.67
Entropy	0.838	67.1	9.25	3.71	0.979	72.9	7.21	4.61	0.960	75.3	7.84	4.52	0.922	77.3	8.51	4.74
Log prob. (ours)	**0.852**	**69.6**	**9.63**	**3.77**	**0.983**	**75.7**	7.21	**4.67**	**0.963**	**76.8**	7.90	**4.58**	**0.924**	**77.6**	**8.59**	**4.76**
Oracle	–	100.0	11.23	4.15	–	100.0	7.35	4.70	–	100.0	8.37	4.67	–	100.0	9.50	5.32

A.7.3 LOOP-AWARE TRAINING.

Table A7 ablates depth exposure (N) and intermediate supervision (mid-CE) in loop-aware training. Enabling loop-aware training with $N = 1$ improves the acceptance length and speedup for all three draft models, with the largest gain for DSpark (5.75 to 8.13 in τ). Increasing N to 3 gives further gains for DFlash and DSpark and comparable results for Domino, whereas $N = 5$ gives no additional improvement despite higher training time and memory. Adding mid-CE consistently lowers the acceptance length and speedup while increasing the training cost, and runs out of memory for DSpark with $N = 5$. We therefore adopt $N = 3$ without mid-CE.

Table A7: **Loop-aware training ablation on depth exposure and intermediate supervision.** We report the 8-benchmark average acceptance length (τ), decoding speedup, training time per iteration (Tr. s/it), and peak GPU memory (Tr. GB) with Qwen3-4B. $N = 0$ disables loop-aware training, and mid-CE applies supervision to intermediate drafting stages. **Bold** indicates the adopted setting.

N	mid-CE	DFlash				Domino				DSpark			
		τ	speedup	Tr. s/it	Tr. GB	τ	speedup	Tr. s/it	Tr. GB	τ	speedup	Tr. s/it	Tr. GB
0		6.33	4.21×	0.136	22.5	7.18	4.33×	0.161	24.4	5.75	3.77×	0.178	32.8
1		7.04	4.52×	0.223	27.6	7.92	4.57×	0.251	34.3	8.13	4.61×	0.292	44.3
3		**7.21**	**4.67×**	**0.299**	**36.2**	**7.90**	**4.58×**	**0.324**	**42.9**	**8.59**	**4.76×**	**0.365**	**51.6**
3	✓	7.07	4.53×	0.321	38.7	7.87	4.54×	0.349	47.6	8.42	4.64×	0.440	65.9
5		7.13	4.55×	0.361	45.0	7.90	4.56×	0.392	52.1	8.52	4.71×	0.427	59.7
5	✓	7.02	4.53×	0.394	50.0	7.89	4.55×	0.436	61.6	*out of memory*			

A.8 Details of the Comparison with Adaptive Draft Length

A.8.1 Decision rules.

BiLD, SVIP, AdaEDL, and LLMCad stop drafting once a statistic of the draft distribution crosses a threshold. BiLD uses the top-1 probability, whereas SVIP and AdaEDL use the entropy. LLMCad uses the confidence accumulated over the draft tokens. AdaEDL and LLMCad also adjust the threshold after each verification according to how many draft tokens were accepted. DDD applies a similar rule to the draft tree of EAGLE-2, checking the summed probability of the sequences in the beam at preset depths. SpecDec++ and DISCO instead train a head that decides after each draft token whether to stop. AdaEAGLE trains a predictor that sets the draft length once, before the drafting stage begins. PACER drafts in blocks of b tokens (b is set to each draft model's draft length) and trains a pre-verification layer. It drafts another block while the mean acceptance that this layer predicts for the latest block remains above a threshold. The confidence scheduler of DSpark chooses how many draft tokens of the full block to verify, using a confidence head trained together with DSpark. The scheduler therefore does not apply to EAGLE-3.

A.8.2 Application to EAGLE-3.

BiLD, SVIP, AdaEDL, LLMCad, SpecDec++, DISCO, and PACER were proposed for a separate small autoregressive LLM as the draft model. EAGLE-3 instead expands its draft tree by one depth per forward pass. We evaluate each rule after every depth on the highest-scoring path of the tree and stop the expansion once the rule stops drafting. PACER runs its pre-verification layer on the same path after every b draft tokens. DDD checks the beam at the preset depths of its paper. We run each rule with the default maximum depth of EAGLE-3 and with the maximum draft length of its paper. We port the fallback policy of BiLD and omit its rollback policy. The rollback policy replaces lossless verification with a rule that can change the output of the target model.

A.8.3 Application to DSpark.

The parallel draft model DSpark computes all k draft tokens of a drafting stage in a single forward pass. Every method except PACER and DDD evaluates its rule on the full block. Only the draft tokens before the position where the rule stops go to verification. For AdaEAGLE, the predicted length sets the number of verified draft tokens. Discarding the remaining draft tokens shortens only the verification. With Qwen3-4B on a single H100 GPU at batch size 1, a verification takes 22.2 ± 0.3 ms for any number of draft tokens from 2 to 16. When a rule does not stop within the block, we also let it perform another drafting stage, up to the maximum draft length of its paper. PACER and DDD instead decide at the end of each drafting stage whether another drafting stage follows. All additional drafting stages use the publicly released weights of DSpark and take the input of Eq. 3. Since DSpark drafts a single chain, the criterion of DDD becomes the sum of the log probabilities of all draft tokens after the anchor token. It differs from our gate function only in accumulating over drafting stages.

A.8.4 TRAINING AND TUNING.

We train the decision heads of SpecDec++, DISCO, AdaEAGLE, and PACER for one epoch on the same data as loop-aware training. Their architecture, inputs, labels, and loss follow the original papers. We keep the other constants of each rule from its paper and sweep its threshold, together with b for PACER. Table 3 reports, for each method, the setting with the highest average speedup.

Table A8: **Prompt templates of the DFlash evaluation code.** Braces mark the fields filled from each sample. Long lines are wrapped for display.

GSM8K, MATH-500, AIME25

```
{problem}
Please reason step by step, and put your final answer within \boxed{}.
```

HumanEval

```
Write a solution to the following problem and make sure that it passes the tests:
```python
{prompt}
```
```

LCB, problems with starter code

```
You are an expert Python programmer. You will be given a question (problem
specification) and will generate a correct Python program that matches the
specification and passes all tests. You will NOT return anything except for the
program

### Question:
{question_content}

### Format: Use the following code structure:
```python
{starter_code}
```

Answer: (use the provided format with backticks)
```

LCB, problems without starter code

```
### Format: Write your code in the following format:
```python
# YOUR CODE HERE
```
```

Alpaca

```
{instruction}

Input:
{input}
```

Table A9: **Acceptance length (τ) and speedup (spd) on each benchmark.** Each cell reports τ / spd×. Avg. is the mean over the eight benchmarks. For each draft model, the first row is the publicly released draft model. "+ Loop" enables the confidence-gated drafting loop, "+ Train" applies one epoch of loop-aware training, and "+ DLoop" applies both. "+ DDTree" and "+ DARTree" apply tree verification. Bold marks the best result in each column for each draft model.

	MATH			CODE			CHAT		
Method	GSM8K	MATH-500	AIME25	HumanEval	MBPP	LCB	MT-Bench	Alpaca	*Avg.*
Qwen3-4B									
EAGLE-3	8.11 / 3.99×	7.98 / 4.02×	7.41 / 3.54×	7.25 / 3.57×	7.25 / 3.49×	6.75 / 3.40×	5.69 / 2.59×	4.91 / 2.17×	6.92 / 3.35×
+ Loop	10.29 / 4.25×	9.68 / 4.32×	9.12 / 3.91×	8.34 / 3.74×	7.92 / 3.58×	7.60 / 3.46×	6.32 / 2.65×	5.05 / **2.27**×	8.04 / 3.52×
+ Train	8.11 / 3.99×	8.02 / 4.06×	7.50 / 3.60×	7.31 / 3.59×	7.32 / 3.49×	6.97 / 3.46×	5.75 / 2.60×	4.97 / 2.17×	6.99 / 3.37×
+ **DLoop**	**14.14 / 4.79**×	**12.25 / 4.87**×	**9.73 / 4.09**×	**9.85 / 3.96**×	**8.94 / 3.81**×	**9.51 / 3.70**×	**7.22 / 2.69**×	**5.36** / 2.25×	**9.63 / 3.77**×
DFlash	6.51 / 4.60×	7.79 / 5.52×	7.26 / 4.94×	6.69 / 4.84×	6.12 / 4.23×	6.97 / 4.88×	4.38 / 2.58×	3.09 / 1.95×	6.10 / 4.19×
+ Loop	6.51 / 4.56×	7.81 / 5.48×	7.27 / 4.94×	6.73 / 4.61×	6.13 / 4.19×	6.98 / 4.94×	4.39 / 2.59×	3.09 / 1.98×	6.11 / 4.16×
+ Train	8.56 / 5.15×	8.31 / 5.63×	7.12 / 4.79×	6.37 / 4.59×	5.98 / 4.11×	6.91 / 4.82×	4.62 / 2.72×	3.20 / 2.05×	6.38 / 4.23×
+ **DLoop**	10.43 / 6.73×	9.99 / 6.46×	8.02 / 5.21×	6.97 / 4.82×	6.32 / 4.26×	7.68 / 5.07×	5.00 / 2.67×	3.24 / 2.09×	7.21 / 4.67×
+ DDTree	9.05 / 6.33×	10.36 / 7.45×	9.58 / 6.42×	9.37 / 6.57×	8.85 / 6.03×	9.47 / 6.40×	6.51 / 3.94×	4.82 / 3.18×	8.50 / 5.79×
+ DDTree + **DLoop**	**14.06 / 8.59**×	**13.94 / 8.52**×	**11.81 / 7.23**×	**10.38 / 6.93**×	**9.48 / 6.07**×	**10.75 / 6.87**×	**7.37 / 4.19**×	**5.17 / 3.43**×	**10.37 / 6.48**×
Domino	10.08 / 6.40×	8.99 / 5.83×	7.48 / 4.67×	7.07 / 4.49×	7.03 / 4.28×	6.86 / 4.39×	5.26 / 2.75×	3.87 / 2.21×	7.08 / 4.38×
+ Loop	10.32 / 6.19×	9.15 / 5.59×	7.55 / 4.62×	7.11 / 4.53×	7.06 / 4.33×	6.90 / 4.37×	5.27 / 2.73×	3.88 / 2.21×	7.15 / 4.32×
+ Train	9.99 / 6.35×	9.05 / 5.79×	7.45 / 4.65×	7.00 / 4.60×	6.95 / 4.26×	6.98 / 4.45×	5.25 / 2.70×	3.84 / 2.20×	7.06 / 4.38×
+ **DLoop**	12.80 / 6.98×	10.72 / 6.31×	7.81 / 4.66×	7.51 / 4.60×	7.33 / 4.38×	7.40 / 4.65×	5.69 / 2.72×	3.95 / 2.38×	7.90 / 4.58×
+ DARTree	12.98 / 6.71×	12.04 / 6.48×	10.31 / 5.27×	9.99 / 5.38×	10.21 / 5.30×	9.31 / 5.09×	7.38 / 3.42×	5.78 / **2.87**×	9.75 / 5.06×
+ DARTree + **DLoop**	**16.65 / 7.65**×	**14.47 / 7.12**×	**11.55 / 5.73**×	**11.08 / 5.58**×	**10.88 / 5.59**×	**10.47 / 5.13**×	**8.14 / 3.50**×	**6.10** / 2.86×	**11.17 / 5.40**×
DSpark	6.58 / 4.55×	6.44 / 4.62×	5.75 / 3.90×	5.73 / 4.05×	5.57 / 3.86×	5.55 / 3.97×	4.48 / 2.82×	3.70 / 2.42×	5.47 / 3.77×
+ Loop	7.20 / 4.62×	6.91 / 4.73×	6.01 / 3.99×	6.03 / 4.08×	5.82 / 3.85×	5.80 / 3.82×	4.63 / 2.80×	3.74 / 2.40×	5.77 / 3.79×
+ Train	6.53 / 4.52×	6.33 / 4.56×	5.63 / 3.86×	5.59 / 4.01×	5.50 / 3.80×	5.66 / 4.01×	4.40 / 2.74×	3.64 / 2.35×	5.41 / 3.73×
+ **DLoop**	13.34 / 6.69×	11.38 / 6.39×	8.34 / **4.97**×	8.31 / 5.00×	7.81 / 4.50×	9.19 / 5.08×	6.27 / 2.95×	4.08 / 2.46×	8.59 / 4.76×
+ DARTree	7.27 / 4.84×	7.04 / 4.79×	6.50 / 4.19×	6.94 / 4.62×	6.77 / 4.32×	6.63 / 4.48×	5.55 / 3.48×	4.90 / 3.03×	6.45 / 4.22×
+ DARTree + **DLoop**	**15.41 / 6.80**×	**13.02 / 6.56**×	**9.38** / 4.93×	**10.45 / 5.71**×	**9.38 / 5.01**×	**10.40 / 5.50**×	**7.66 / 3.65**×	**5.38 / 3.04**×	**10.14 / 5.15**×
Qwen3-8B									
EAGLE-3	7.87 / 3.94×	7.81 / 3.82×	7.10 / 3.58×	6.79 / 3.47×	6.85 / 3.29×	6.55 / 3.14×	5.24 / 2.40×	4.72 / 2.13×	6.61 / 3.22×
+ Loop	9.43 / 4.30×	9.33 / 4.10×	8.16 / 3.89×	7.32 / 3.53×	7.12 / 3.37×	7.05 / 3.21×	5.48 / **2.42**×	4.80 / **2.23**×	7.34 / 3.38×
+ Train	7.90 / 3.84×	7.80 / 3.82×	6.97 / 3.55×	6.80 / 3.39×	6.86 / 3.23×	6.53 / 3.12×	5.26 / 2.37×	4.71 / 1.87×	6.60 / 3.15×
+ **DLoop**	**11.76 / 4.62**×	**10.98 / 4.29**×	**8.61 / 4.01**×	**7.95 / 3.66**×	**7.77 / 3.45**×	**7.77 / 3.28**×	**6.03** / 2.51×	**4.90** / 2.25×	**8.22 / 3.51**×
DFlash	6.55 / 4.68×	7.93 / 5.50×	7.18 / 5.22×	6.56 / 4.47×	5.98 / 4.05×	7.16 / 4.82×	4.23 / 2.45×	3.09 / 1.99×	6.08 / 4.15×
+ Loop	6.60 / 4.64×	8.11 / 5.29×	7.59 / 5.45×	6.60 / 4.61×	6.04 / 4.11×	7.29 / 4.81×	4.27 / 2.46×	3.10 / 2.02×	6.20 / 4.17×
+ Train	8.64 / 5.48×	8.69 / 5.86×	7.09 / 5.08×	6.50 / 4.60×	6.06 / 4.19×	7.17 / 4.76×	4.51 / 2.59×	3.19 / 2.09×	6.48 / 4.33×
+ **DLoop**	10.73 / 6.78×	10.79 / 6.60×	7.83 / 5.46×	7.17 / 4.78×	6.36 / 4.27×	8.12 / 5.06×	4.93 / 2.65×	3.24 / 2.08×	7.40 / 4.71×
+ DDTree	9.12 / 6.55×	10.46 / 7.27×	9.39 / 6.65×	9.33 / 6.73×	8.66 / 5.93×	9.75 / 6.66×	6.30 / 3.81×	4.82 / 3.34×	8.48 / 5.87×
+ DDTree + **DLoop**	**14.24 / 9.08**×	**14.14 / 8.68**×	**10.54 / 7.31**×	**10.38 / 6.97**×	**9.10 / 6.26**×	**11.07 / 7.11**×	**7.18 / 4.06**×	**5.07 / 3.52**×	**10.22 / 6.62**×
Domino	9.98 / 6.39×	9.43 / 5.80×	7.72 / 5.05×	7.33 / 4.75×	7.11 / 4.37×	7.08 / 4.35×	5.19 / 2.70×	3.90 / 2.32×	7.22 / 4.47×
+ Loop	10.56 / 6.47×	9.87 / 5.87×	7.76 / 5.04×	7.44 / 4.68×	7.17 / 4.44×	7.20 / 4.45×	5.26 / 2.76×	3.93 / 2.35×	7.40 / 4.51×
+ Train	10.03 / 6.57×	9.47 / 5.80×	7.73 / 5.13×	7.36 / 4.86×	7.06 / 4.43×	7.10 / 4.33×	5.19 / 2.70×	3.90 / 2.35×	7.23 / 4.52×
+ **DLoop**	13.42 / 7.30×	11.70 / 6.12×	8.32 / 5.52×	7.97 / 4.93×	7.33 / 4.46×	7.78 / 4.54×	5.76 / 2.77×	4.06 / 2.45×	8.29 / 4.76×
+ DARTree	12.94 / 6.97×	12.19 / 6.32×	10.45 / 5.65×	10.35 / 5.61×	10.10 / 5.26×	9.73 / 5.00×	7.38 / 3.31×	5.80 / 2.89×	9.87 / 5.13×
+ DARTree + **DLoop**	**17.23 / 7.80**×	**15.44 / 7.00**×	**12.57 / 6.19**×	**11.93 / 5.87**×	**11.06 / 5.63**×	**11.16 / 5.26**×	**8.25 / 3.43**×	**6.17 / 2.97**×	**11.73 / 5.52**×
DSpark	6.64 / 4.74×	6.55 / 4.51×	5.79 / 4.14×	5.88 / 4.08×	5.72 / 3.81×	5.83 / 3.98×	4.41 / 2.72×	3.85 / 2.49×	5.58 / 3.81×
+ Loop	7.27 / 4.86×	7.10 / 4.47×	6.20 / 4.28×	6.16 / 4.16×	5.95 / 3.87×	6.24 / 3.95×	4.54 / 2.78×	3.90 / 2.55×	5.92 / 3.87×
+ Train	6.60 / 4.67×	6.42 / 4.42×	5.70 / 4.05×	5.76 / 4.05×	5.60 / 3.82×	5.81 / 4.04×	4.37 / 2.63×	3.77 / 2.49×	5.50 / 3.77×
+ **DLoop**	13.83 / 6.94×	12.66 / 6.08×	8.90 / **5.58**×	9.03 / 5.22×	7.99 / 4.58×	9.48 / 4.70×	6.29 / 2.91×	4.49 / 2.57×	9.08 / 4.82×
+ DARTree	7.43 / 4.86×	7.21 / 4.50×	6.61 / 4.37×	7.04 / 4.58×	6.89 / 4.35×	6.85 / 4.28×	5.60 / 3.39×	5.24 / **3.31**×	6.61 / 4.21×
+ DARTree + **DLoop**	**14.20 / 7.25**×	**13.00 / 6.31**×	**9.73** / 5.51×	**10.60 / 5.85**×	**9.23 / 5.00**×	**10.65 / 5.42**×	**7.71 / 3.60**×	**5.74 / 3.31**×	**10.11 / 5.28**×
Qwen3.5-9B									
MTP	3.59 / 1.76×	3.59 / 1.80×	3.45 / 1.64×	3.60 / 1.71×	3.30 / 1.53×	3.27 / 1.49×	2.98 / 1.30×	2.78 / 1.13×	3.32 / 1.55×
+ Loop	6.04 / 2.19×	5.82 / 2.20×	5.12 / 1.91×	6.73 / 2.25×	4.35 / 1.69×	4.43 / 1.63×	4.02 / 1.38×	3.12 / 1.16×	4.96 / 1.80×
+ Train	3.72 / 1.87×	3.66 / 1.85×	3.45 / 1.63×	3.66 / 1.80×	3.42 / 1.65×	3.29 / 1.53×	2.99 / 1.30×	2.81 / 1.16×	3.37 / 1.60×
+ **DLoop**	**10.46 / 3.13**×	**8.39 / 2.75**×	**5.92 / 2.09**×	**9.47 / 2.83**×	**5.72 / 2.09**×	**5.59 / 1.87**×	**4.63 / 1.44**×	**3.41 / 1.22**×	**6.70 / 2.18**×
Gemma-4-12B									
MTP	5.45 / 3.57×	5.48 / 3.71×	5.20 / 3.38×	5.63 / 3.75×	4.72 / 3.13×	5.01 / 3.24×	3.91 / 2.41×	3.51 / **2.16**×	4.86 / 3.17×
+ Loop	7.14 / 3.28×	7.11 / 3.30×	6.64 / 3.02×	7.88 / 3.38×	5.49 / 2.96×	6.06 / 2.88×	4.52 / 2.31×	3.68 / 2.10×	6.06 / 2.90×
+ Train	5.83 / 3.81×	5.65 / 3.77×	5.13 / 3.34×	5.74 / 3.76×	4.90 / 3.40×	4.99 / 3.23×	3.88 / 2.36×	3.45 / 2.07×	4.95 / 3.22×
+ **DLoop**	**10.96 / 4.80**×	**9.23 / 4.48**×	**6.99 / 3.69**×	**10.61 / 4.47**×	**6.42 / 3.80**×	**6.99 / 3.57**×	**4.97 / 2.45**×	**3.82** / 2.15×	**7.50 / 3.68**×

Table A10: **Acceptance length (τ) and speedup (spd) on each benchmark with sampling at temperature $T = 1$.** The rows and the notation follow Table A9. Verification follows the sampling rule of the official implementation of each draft model. The gate threshold is selected at $T = 1$ in the same way as at $T = 0$. Bold marks the best result in each column for each draft model.

Method	MATH			CODE			CHAT		*Avg.*
	GSM8K	MATH-500	AIME25	HumanEval	MBPP	LCB	MT-Bench	Alpaca	
Qwen3-4B									
EAGLE-3	7.83 / 3.80×	7.36 / 3.59×	5.97 / 2.98×	6.94 / 3.39×	6.99 / 3.37×	6.56 / 3.16×	5.27 / 2.32×	4.69 / 1.80×	6.45 / 3.05×
+ Loop	9.71 / 4.02×	8.68 / 3.76×	6.54 / 3.13×	8.01 / 3.55×	7.53 / 3.49×	6.95 / 3.12×	5.68 / 2.32×	4.85 / 2.07×	7.24 / 3.18×
+ Train	7.91 / 3.86×	7.32 / 3.55×	6.18 / 3.09×	6.95 / 3.42×	7.07 / 3.50×	6.75 / 3.24×	5.43 / 2.43×	4.79 / 2.07×	6.55 / 3.14×
+ **DLoop**	**12.86 / 4.46**×	**10.95 / 4.09**×	**7.08 / 3.23**×	**9.10 / 3.76**×	**8.63 / 3.77**×	**8.30 / 3.42**×	**6.55 / 2.48**×	**5.05 / 2.10**×	**8.56 / 3.41**×
DFlash	6.03 / 4.32×	6.59 / 4.61×	4.90 / 3.29×	6.00 / 4.16×	5.61 / 3.99×	6.40 / 4.35×	4.01 / 2.37×	2.91 / 1.81×	5.31 / 3.61×
+ Loop	6.03 / 4.23×	6.83 / 4.50×	5.11 / 3.59×	6.06 / 4.19×	5.68 / 3.84×	6.66 / 4.46×	4.07 / 2.31×	2.92 / 1.84×	5.42 / 3.62×
+ Train	7.46 / 5.32×	6.91 / 4.75×	4.72 / 3.32×	5.86 / 4.19×	5.58 / 3.96×	6.59 / 4.46×	4.18 / 2.42×	2.99 / 1.95×	5.54 / 3.80×
+ **DLoop**	8.75 / 5.53×	7.87 / 5.07×	5.03 / 3.42×	6.24 / 4.39×	5.67 / 3.95×	6.69 / 4.35×	4.46 / 2.39×	3.03 / 1.99×	5.97 / 3.89×
+ DDTree	8.63 / 6.01×	9.27 / 6.16×	7.45 / **4.99**×	8.71 / **6.02**×	8.39 / **5.76**×	9.00 / 5.79×	6.04 / 3.51×	4.66 / 2.88×	7.77 / 5.14×
+ DDTree + **DLoop**	**11.58 / 7.46**×	**10.84 / 6.75**×	**7.74** / 4.90×	**9.29** / 5.99×	**8.52** / 5.73×	**9.43 / 6.12**×	**6.61 / 3.66**×	**4.92 / 3.12**×	**8.62 / 5.47**×
Domino	7.98 / 4.69×	6.87 / 3.88×	4.19 / 2.51×	5.66 / 3.36×	5.50 / 3.20×	5.79 / 3.28×	4.11 / 1.98×	3.11 / 1.64×	5.40 / 3.07×
+ Loop	8.36 / 4.65×	7.08 / 3.83×	4.35 / 2.56×	5.75 / 3.34×	5.53 / 3.20×	5.83 / 3.23×	4.10 / 1.97×	3.13 / 1.64×	5.51 / 3.05×
+ Train	7.96 / 4.65×	6.87 / 3.92×	4.25 / 2.52×	5.65 / 3.36×	5.47 / 3.17×	5.60 / 3.27×	3.97 / 1.91×	3.07 / 1.64×	5.36 / 3.05×
+ **DLoop**	9.58 / 4.98×	7.76 / 3.95×	4.45 / 2.59×	5.92 / 3.39×	5.60 / 3.17×	5.86 / 3.37×	4.16 / 1.89×	3.14 / 1.62×	5.81 / 3.12×
+ DARTree	11.50 / 5.96×	9.93 / 5.13×	7.28 / 3.84×	9.14 / 4.86×	9.25 / 4.83×	8.93 / 4.57×	6.75 / 3.06×	5.41 / 2.55×	8.52 / 4.35×
+ DARTree + **DLoop**	**14.12 / 6.55**×	**11.62 / 5.36**×	**8.10 / 4.04**×	**10.15 / 5.03**×	**10.07 / 5.10**×	**9.45 / 4.69**×	**7.34 / 3.14**×	**5.69 / 2.57**×	**9.57 / 4.56**×
DSpark	6.39 / 4.44×	6.07 / 4.20×	5.16 / 3.67×	5.56 / 3.84×	5.47 / 3.63×	5.34 / 3.72×	4.25 / 2.63×	3.52 / 2.28×	5.22 / 3.55×
+ Loop	7.02 / 4.44×	6.52 / 4.15×	5.14 / 3.48×	5.78 / 3.73×	5.68 / 3.55×	5.56 / 3.59×	4.45 / 2.63×	3.63 / 2.28×	5.47 / 3.48×
+ Train	6.32 / 4.45×	5.93 / 4.14×	4.94 / 3.55×	5.41 / 3.71×	5.33 / 3.67×	5.39 / 3.64×	4.21 / 2.59×	3.53 / 2.27×	5.13 / 3.50×
+ **DLoop**	**11.71** / 6.07×	**10.40 / 5.37**×	6.45 / 3.97×	7.61 / 4.41×	7.03 / 4.12×	7.52 / 4.09×	5.51 / 2.72×	3.95 / 2.30×	7.52 / 4.13×
+ DARTree	7.04 / 4.71×	6.55 / 4.28×	5.45 / 3.55×	6.68 / 4.38×	6.55 / 4.18×	6.46 / 4.22×	5.22 / 3.09×	4.62 / 2.73×	6.07 / 3.89×
+ DARTree + **DLoop**	11.31 / **6.28**×	9.81 / 5.26×	**6.53 / 4.05**×	**9.11 / 5.26**×	**8.08 / 4.83**×	**8.94 / 4.87**×	**6.64 / 3.26**×	**5.05 / 2.90**×	**8.18 / 4.59**×
Qwen3-8B									
EAGLE-3	7.54 / 3.58×	7.15 / 3.41×	5.72 / 2.78×	6.22 / 3.00×	6.40 / 3.10×	6.37 / 2.96×	4.91 / 2.17×	4.45 / 1.83×	6.10 / 2.85×
+ Loop	8.87 / 3.82×	8.03 / 3.56×	6.12 / 2.92×	6.63 / 3.07×	6.58 / 3.12×	6.69 / 3.01×	5.10 / 2.17×	4.52 / 1.97×	6.57 / 2.96×
+ Train	7.53 / 3.61×	7.14 / 3.54×	5.61 / 2.80×	6.28 / 3.03×	6.45 / 3.15×	6.37 / 3.00×	4.89 / 2.17×	4.40 / 1.95×	6.08 / 2.91×
+ **DLoop**	**10.37 / 4.09**×	**9.22 / 3.87**×	**6.35 / 2.95**×	**7.06 / 3.19**×	**7.06 / 3.27**×	**6.99 / 3.04**×	**5.43 / 2.22**×	**4.54 / 1.99**×	**7.13 / 3.08**×
DFlash	5.95 / 4.09×	6.47 / 4.50×	4.84 / 3.28×	5.53 / 3.92×	5.23 / 3.69×	6.79 / 4.38×	3.85 / 2.26×	2.99 / 1.88×	5.21 / 3.50×
+ Loop	5.95 / 4.09×	6.47 / 4.27×	4.98 / 3.23×	5.58 / 3.89×	5.27 / 3.63×	6.86 / 4.28×	3.77 / 2.16×	2.97 / 1.88×	5.23 / 3.43×
+ Train	7.36 / 4.96×	6.68 / 4.54×	4.80 / 3.35×	5.58 / 3.93×	5.28 / 3.71×	6.67 / 4.39×	4.00 / 2.36×	3.02 / 1.98×	5.42 / 3.65×
+ **DLoop**	8.88 / 5.38×	8.22 / 4.85×	4.93 / 3.22×	5.99 / 4.08×	5.45 / 3.59×	7.41 / 4.46×	4.41 / 2.36×	3.07 / 1.91×	6.05 / 3.73×
+ DDTree	8.60 / 5.81×	9.12 / 6.22×	7.20 / **4.74**×	8.23 / **5.65**×	7.93 / 5.43×	9.27 / 6.10×	5.78 / **3.53**×	4.62 / 2.90×	7.59 / 5.05×
+ DDTree + **DLoop**	**11.49 / 7.16**×	**10.22 / 6.44**×	**7.26** / 4.62×	**8.61** / 5.63×	**7.96 / 5.52**×	**9.88 / 6.29**×	**6.13** / 3.52×	**4.78 / 3.12**×	**8.29 / 5.29**×
Domino	8.08 / 4.74×	6.76 / 3.89×	4.33 / 2.59×	5.43 / 3.29×	5.18 / 3.01×	6.06 / 3.37×	3.90 / 1.91×	3.04 / 1.66×	5.35 / 3.06×
+ Loop	8.26 / 4.53×	6.88 / 3.85×	4.46 / 2.60×	5.38 / 3.20×	5.22 / 3.00×	6.10 / 3.33×	3.97 / 1.90×	3.00 / 1.66×	5.41 / 3.01×
+ Train	7.94 / 4.45×	6.76 / 3.91×	4.46 / 2.66×	5.41 / 3.28×	5.21 / 3.07×	6.14 / 3.39×	3.92 / 1.87×	2.97 / 1.67×	5.35 / 3.04×
+ **DLoop**	9.09 / 4.57×	7.57 / 3.89×	4.48 / 2.62×	5.54 / 3.25×	5.34 / 3.09×	6.33 / 3.39×	4.10 / 1.87×	2.99 / 1.67×	5.68 / 3.04×
+ DARTree	11.16 / 5.68×	9.59 / 4.91×	7.00 / 3.62×	8.55 / 4.44×	8.91 / 4.56×	9.24 / 4.65×	6.33 / 2.91×	5.23 / 2.52×	8.25 / 4.16×
+ DARTree + **DLoop**	**13.94 / 6.16**×	**11.60 / 5.27**×	**7.42 / 3.68**×	**9.62 / 4.68**×	**9.48 / 4.69**×	**10.19 / 4.84**×	**7.06 / 2.93**×	**5.60 / 2.60**×	**9.36 / 4.36**×
DSpark	6.44 / 4.39×	6.10 / 4.25×	5.07 / 3.43×	5.62 / 3.89×	5.44 / 3.67×	5.70 / 3.86×	4.18 / 2.65×	3.67 / 2.33×	5.28 / 3.56×
+ Loop	7.10 / 4.42×	6.54 / 4.18×	5.16 / 3.34×	5.82 / 3.76×	5.76 / 3.66×	6.02 / 3.71×	4.33 / 2.59×	3.75 / 2.29×	5.56 / 3.49×
+ Train	6.40 / 4.30×	5.99 / 4.11×	4.95 / 3.45×	5.44 / 3.71×	5.35 / 3.63×	5.54 / 3.67×	4.14 / 2.64×	3.63 / 2.28×	5.18 / 3.47×
+ **DLoop**	11.53 / 5.91×	10.04 / 5.26×	6.30 / 3.82×	7.70 / 4.43×	7.02 / 4.10×	8.59 / 4.29×	5.63 / 2.72×	4.04 / 2.33×	7.61 / 4.11×
+ DARTree	7.21 / 4.57×	6.72 / 4.31×	5.61 / 3.54×	6.50 / 4.00×	6.63 / 4.16×	6.77 / 4.12×	5.17 / 3.11×	4.97 / **2.97**×	6.20 / 3.85×
+ DARTree + **DLoop**	**12.09 / 6.04**×	**10.64 / 5.39**×	**6.71 / 3.85**×	**8.98 / 4.82**×	**8.19 / 4.58**×	**10.00 / 5.03**×	**6.53 / 3.21**×	**5.32** / 2.89×	**8.56 / 4.48**×
Qwen3.5-9B									
MTP	3.50 / 1.73×	3.45 / 1.66×	3.23 / 1.52×	3.43 / 1.66×	3.09 / 1.38×	3.05 / 1.38×	2.69 / 1.09×	2.53 / 1.03×	3.12 / 1.43×
+ Loop	5.64 / 2.08×	5.40 / 1.98×	4.59 / 1.77×	6.06 / 2.07×	3.95 / 1.51×	3.91 / 1.46×	3.39 / 1.13×	2.84 / 1.06×	4.47 / 1.63×
+ Train	3.57 / 1.76×	3.51 / 1.73×	3.26 / 1.53×	3.51 / 1.73×	3.16 / 1.45×	3.08 / 1.39×	2.69 / 1.08×	2.54 / 1.05×	3.17 / 1.47×
+ **DLoop**	**9.56 / 2.83**×	**7.43 / 2.40**×	**5.13 / 1.85**×	**7.87 / 2.39**×	**5.08 / 1.78**×	**4.88 / 1.62**×	**4.01 / 1.20**×	**3.08 / 1.10**×	**5.88 / 1.90**×
Gemma-4-12B									
MTP	5.28 / 3.49×	5.21 / 3.56×	4.86 / 3.19×	5.35 / 3.61×	4.50 / 2.96×	4.61 / 2.94×	3.82 / 2.31×	3.44 / 2.07×	4.63 / 3.02×
+ Loop	6.75 / 3.18×	6.62 / 3.17×	5.83 / 2.80×	7.35 / 3.27×	5.17 / 2.79×	5.49 / 2.60×	4.40 / 2.18×	3.57 / 2.02×	5.65 / 2.75×
+ Train	5.59 / 3.76×	5.33 / 3.62×	4.75 / 3.09×	5.42 / 3.59×	4.61 / 2.92×	4.64 / 2.96×	3.76 / 2.26×	3.38 / 2.07×	4.69 / 3.03×
+ **DLoop**	**10.01 / 4.47**×	**8.27 / 4.18**×	**5.96 / 3.29**×	**9.61 / 4.19**×	**5.84 / 3.20**×	**6.25 / 3.20**×	**4.78 / 2.33**×	**3.69 / 2.09**×	**6.80 / 3.37**×